



المدرسة الوطنية للعلوم
التطبيقية - مراكش
ÉCOLE NATIONALE DES SCIENCES
APPLIQUÉES - MARRAKECH



وكالة الحوض المائي لتانسيفت
Agence de Bassin Hydraulique du Tensift

Université Cadi Ayyad

École Nationale des Sciences Appliquées de Marrakech

Filière Génie Cyber-Défense et Systèmes de

Télécommunications Embarqués (GCDSTE)

Rapport De Projet du stage

Détection d’Intrusion et Analyse des Menaces via Honeypots

Réalisé par :

- Sayf eddine Laamri

Encadré par :

- Hicham Errafiy
- Abdelkarim Damhari
- Fatiha Choukri

Date : 30/07/2025

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce projet de stage au sein de l'Agence du Bassin Hydraulique du Tensift (ABHT).

Je remercie tout particulièrement Monsieur Abdelkarim Damhari, Monsieur Hicham Errafiy et Madame Fatiha Choukri, mes encadrants de stage au sein du Département Informatique, pour leur encadrement bienveillant, leur disponibilité, ainsi que la richesse de leurs conseils tout au long de cette expérience. Leur accompagnement m'a permis de consolider mes compétences et de mener à bien ce projet dans des conditions optimales.

Mes remerciements s'adressent également à toute l'équipe du Département Informatique de l'ABHT, pour leur accueil chaleureux, leur collaboration et leur partage de savoirs. Leur implication et leur professionnalisme ont joué un rôle essentiel dans la réussite de ce stage, notamment dans le domaine de la cybersécurité des réseaux et des systèmes d'information.

Je n'oublie pas toutes les personnes, collègues, amis ou formateurs, qui m'ont soutenu ou aidé d'une manière ou d'une autre pendant ce stage, que ce soit par leur appui technique, leurs conseils ou leur encouragement.

Enfin, je remercie sincèrement ma famille et mes proches pour leur soutien constant, leur patience et leur motivation précieuse tout au long de cette période de formation et de travail.

À toutes et à tous, merci pour votre confiance, votre accompagnement et votre bienveillance.

RÉSUMÉ

Ce projet a pour objectif de concevoir et de mettre en place une solution de détection et de prévention des intrusions en intégrant les outils Suricata et Wazuh. Suricata agit comme un moteur de surveillance réseau performant, capable d'analyser le trafic en temps réel et de détecter diverses menaces. Wazuh, quant à lui, assure la collecte, la corrélation et l'analyse centralisée des événements de sécurité, permettant une détection avancée des attaques et une gestion efficace des alertes.

Pour simuler des scénarios d'attaque réalistes, une application web vulnérable (DVWA) est utilisée comme honeypot, attirant ainsi les attaquants sans mettre en danger les systèmes réels. Cette combinaison permet non seulement de détecter les intrusions, mais aussi d'observer les techniques utilisées par les attaquants à des fins d'analyse et de compréhension.

Le projet adopte une architecture modulaire et évolutive, facilitant l'installation, la configuration et le suivi des événements. Il constitue un outil à la fois pédagogique et pratique pour l'apprentissage des concepts fondamentaux de la cybersécurité moderne.

ABSTRACT :

This project aims to design and implement a comprehensive intrusion detection and prevention framework by integrating Suricata and Wazuh. Suricata serves as a high-performance IDS/IPS and network security monitoring engine capable of analyzing real-time traffic and detecting various types of attacks. Wazuh complements this by collecting, correlating, and centralizing security events, enabling advanced threat detection, log analysis, and alerting.

To simulate real-world attack scenarios, a vulnerable web application (DVWA) is deployed as a honeypot. This setup allows the system to capture malicious behavior and test detection capabilities under realistic conditions. The integration of these components provides both visibility and defensive mechanisms against intrusions, while enabling detailed forensic analysis.

The project follows a modular and scalable design, facilitating ease of configuration, monitoring, and expansion. It serves as an educational and practical tool for exploring network security monitoring, threat detection, and log correlation in modern cybersecurity environments.

ABRÉVIATIONS :

GCDSTE	Génie Cyber Défense et Systèmes de Télécommunications Embarqués
ENSA	Ecole Nationale des Sciences Appliquées
IDS	Intrusion Detection System
IP	Internet Protocol
IPS	Intrusion Prevention System
XSS	Cross-Site Scripting
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
Firewall	Pare-feu
JSON	JavaScript Object Notation
SOC	Security Operations Center
DGSSI	Direction Générale de la Sécurité des Systèmes d'Information
CNDP	Commission Nationale de Protection des Données Personnelles
ANRT	Agence Nationale de Réglementation des Télécommunications
FIM	File Integrity Monitoring (Surveillance d'intégrité des fichiers
VM	Virtual Machine (Machine virtuelle)
DVWA	Damn Vulnerable Web Application
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol (Protocole d'envoi d'e-mails)

TABLE DES FIGURES

Figure 1 : Logo de ABHT.....	16
Figure 2 : Logo de ENSA.....	18
Figure 3 : Logo de GCDSTE.....	19
Figure 4 : Logo de SURICATA.....	22
Figure 5 : Logo de DVWA	23
Figure 6 : Logo de Wazuh.....	24
Figure 7 : Logo de AbuseIPDB.....	25
Figure 8 : Logo de VirusTotal	26
Figure 9 : Logo de IPTABLES.....	27
Figure 10 : Logo de VMware	29
Figure 11 : Installation de Suricata.....	32
Figure 12 : Fichier de logs EVE (JSON)	34
Figure 13 : Téléchargement des règles.....	35
Figure 14 : Redémarrage de Suricata.....	36
Figure 15 : Règles personnalisées	36
Figure 16 : Fichier de log fast.log (Suricata).....	37
Figure 17 : DVWA interface.....	38
Figure 18 : Clonage du projet DVWA.....	39
Figure 19 : Attaques simulées sur honeypot.....	40
Figure 20 : Installation de Wazuh server	45
Figure 21 : Démarrage de Wazuh server.....	46
Figure 22 : Installation du Wazuh Agent.....	46
Figure 23 : Configuration de l'agent	47
Figure 24 : Démarrage de Wazuh agent	48
Figure 25 : Interface graphique de Wazuh server	50
Figure 26 : Présentation du Dashboard Wazuh.....	51
Figure 27 : Interface des événements de l'agent.....	54
Figure 28 : AbuseIPDB site web.....	57
Figure 29 : Script d'AbuseIPDB	58

Figure 30 : Test de script d'AbuseIPDB	59
Figure 31 : Schéma du travail de l'API AbuseIPDB	59
Figure 32 : Blocage automatique avec iptables.....	60
Figure 33 : VirusTotal site web.....	61
Figure 34 : Intégration de VirusTotal dans Wazuh	62
Figure 35 : Les répertoires à surveiller	63
Figure 36 : Script de suppression des fichiers malveillants	65
Figure 37 : Script pour envoyer un e-mail.....	69
Figure 38 : E-mail reçu.....	70
Figure 39 : Vérification des configurations du PC.....	71
Figure 40 : Installation de l'antivirus Kaspersky	72
Figure 41 : Vérification des alertes des machines connectées	73
Figure 42 : L'antivirus en tant que pare-feu.....	74

TABLE DES MATIÈRES

1. Remerciements	3
2. Résumé	4
3. Abstract	5
4. Abréviations.....	6
5. Tableau des Figures	7
6. Introduction Générale	G
6.1. Présentation de la cybersécurité.....	10
6.2. Cadre légal et réglementaire de la cybersécurité au Maroc	11
7. Missions et Tâches Réalisées	15
8. Chapitre 21 :Contexte Général.....	17
8.1. Introduction	17
8.2. Présentation de l'entreprise : Agence du Bassin Hydraulique du Tensift (ABHT)	
8.2.1. Mission principale	18
8.2.2. Vision de l'agence	18
8.2.3. Le département informatique	19
8.3. Contexte de mon stage	20
8.4. Présentation de l'ENSA.....	21
8.5. Présentation de la filière GCDSTE	22
G. Chapitre 2 : Étude des Outils Utilisés.....	23
9.1. Suricata : IDS/IPS performant.....	23
9.2. Honeypot DVWA.....	24
9.3. Wazuh : Plateforme SIEM pour surveillance et détection d'anomalies	25
9.4. AbuseIPDB : Base de données d'adresses IP malveillantes	26
9.5. VirusTotal : API	27
9.6. iptables : Le pare-feu natif de Linux	28
9.7. Environnement de virtualisation : VMware Workstation.....	29
10. Chapitre 3 : Implémentation et Réalisation	31
10.1. Introduction	31
10.2. Installation et configuration de Suricata	
10.2.1. Configuration de base	33
10.2.2. Configuration des logs.....	34
10.2.3. Gestion et configuration des règles Suricata.....	35
10.2.4. Résultats de l'analyse.....	37
10.3. Déploiement du honeypot DVWA pour la détection d'attaques	
10.3.1. Configuration du honeypot	39
10.3.2. Attaques simulées.....	40
10.4. Le mode IDS/IPS avec Suricata et Wazuh	

10.4.1. Exemples de règles Suricata	42
10.4.2. Vérification du bon fonctionnement des règles	43
10.4.3. Mode IPS et complémentarité avec iptables.....	44
10.5. Intégration de Wazuh Agent et Manager	
10.5.1. Objectif de Wazuh dans notre projet IDS.....	45
10.5.2. Installation du Wazuh Manager.....	46
10.5.3. Installation et configuration du Wazuh Agent	47
10.5.4. Intégration avec Suricata	48
10.5.5. Visualisation des alertes dans le dashboard.....	49
10.5.6. Résultat des tests.....	50
10.6. Présentation du Dashboard Wazuh	
10.6.1. Agents Summary	52
10.6.2. Last 24 Hours Alerts	52
10.6.3. Endpoint Security	53
10.6.4. Threat Intelligence.....	53
10.6.5. Security Operations.....	54
10.6.6. Cloud Security.....	54
10.7. Analyse de la vue "Dashboard" de Wazuh (agent : Kali)	55
10.8. Intégration de l'API AbuseIPDB à notre IDS	
10.8.1. Blocage automatique avec iptables	59
10.9. Intégration de l'API VirusTotal dans Wazuh et Suricata.....	60
10.10. Surveillance des fichiers critiques (File Integrity Monitoring - FIM)	
10.10.1. Configuration dans Wazuh Agent.....	63
10.10.2. Protection contre les menaces	64
10.10.3. Script de suppression automatique des fichiers malveillants	65
10.11. Rôle du pare-feu iptables dans le projet IDS	
10.11.1. Intégration dans notre architecture	67
10.11.2. Complémentarité avec Suricata et Wazuh	68
10.12. Intégration de l'envoi d'e-mails.....	69
10.13. Mise en place de contrôles de sécurité essentiels sur un poste utilisateur	
10.13.1. Vérification de la version de Windows installée	71
10.13.2. Installation et configuration de l'antivirus Kaspersky.....	72
10.13.3. Compréhension du rôle de l'antivirus en tant que pare-feu	73
11. Compétences Acquises et Défis Rencontrés	74
11.1. Compétences Acquises.....	74
11.2. Défis Rencontrés.....	75
12. Chapitre 4 :Migration et Reconfiguration	77
12.1. Clonage d'une VM	78
12.2. Migration	80
13. Clés d'Accès	82
14. Références.....	76
15. Conclusion Générale	77

INTRODUCTION GÉNÉRALE

Dans un monde de plus en plus interconnecté, les systèmes d'information occupent une place centrale dans le fonctionnement quotidien des organisations. Cette dépendance accrue au numérique rend les structures particulièrement vulnérables face aux cybermenaces, dont la fréquence et la sophistication ne cessent de croître. Dans ce contexte, la cybersécurité s'impose comme un enjeu stratégique majeur, visant à garantir la confidentialité, l'intégrité et la disponibilité des données et des infrastructures informatiques.

Le présent rapport rend compte du stage que j'ai effectué au sein de l'Agence du Bassin Hydraulique du Tensift (ABHT), du 01/07/2025 au 31/07/2025. Ce stage a porté sur la conception et la mise en place d'un système de détection et de prévention d'intrusion (IDS/IPS) basé sur Suricata, intégré à la plateforme de supervision Wazuh, et enrichi par l'API VirusTotal. L'objectif principal était de mettre en œuvre une solution capable de surveiller en temps réel le trafic réseau, identifier des comportements malveillants et centraliser les alertes dans un tableau de bord unifié pour une réaction rapide et automatisée.

Ce document expose dans un premier temps le contexte institutionnel et les fondements de la cybersécurité, avant de détailler les phases de conception, de configuration et de validation du système, ainsi que les enseignements techniques et professionnels tirés de cette expérience.

Présentation de la cybersécurité

La cybersécurité désigne l'ensemble des pratiques, technologies, politiques et moyens mis en œuvre pour protéger les systèmes d'information, les réseaux et les données contre les menaces numériques, qu'elles soient accidentelles ou malveillantes. Elle s'applique à divers domaines tels que la sécurité réseau, la protection des données, la gestion des accès, ou encore la surveillance des vulnérabilités.

Les attaques informatiques se manifestent sous de nombreuses formes : malwares, ransomwares, hameçonnage (phishing), attaques par déni de service (DDoS), ou encore exploitation de failles logicielles. Leur impact peut être considérable, allant de l'interruption de service à la fuite de données sensibles, en passant par la dégradation de l'image de l'organisation ou des pertes économiques.

Dès lors, la cybersécurité n'est plus une simple contrainte technique ; elle devient un levier essentiel de résilience, de confiance numérique et de gouvernance stratégique dans les secteurs public comme privé. Les outils modernes comme les IDS/IPS, les SIEM ou encore les solutions d'analyse comportementale sont désormais indispensables pour faire face à ces enjeux.

Cadre légal et réglementaire de la cybersécurité au Maroc

Consciente de l'importance de la cybersécurité dans la protection du patrimoine numérique national, le Maroc a mis en place un cadre juridique et institutionnel dédié, piloté par la Direction Générale de la Sécurité des Systèmes d'Information (DGSSI), relevant de l'Administration de la Défense Nationale.

La DGSSI est chargée de définir et de mettre en œuvre la stratégie nationale de cybersécurité. Elle accompagne les administrations, entreprises publiques et infrastructures critiques dans la mise en œuvre de mesures de sécurité conformes aux normes internationales.

Parmi les textes de référence encadrant la cybersécurité au Maroc, on retrouve :

- **Loi n° 05-20 : Cadre national de la cybersécurité**

Adoptée en 2020, cette loi constitue la pierre angulaire de la politique nationale de cybersécurité. Elle définit :

- Les Systèmes d'Information d'Importance Vitale (SIIV) et les obligations qui leur sont imposées.
- Les normes de sécurité à respecter pour prévenir, détecter et gérer les incidents de sécurité.
- Le rôle de la DGSSI dans le contrôle, l'homologation des systèmes critiques et l'accompagnement technique.

- **Loi n° 43-20 : Services de confiance pour les transactions électroniques**

Promulguée pour renforcer la confiance dans le numérique, cette loi encadre :

- L'usage des certificats électroniques,
- La signature électronique qualifiée,
- Les prestataires de services de confiance (PSCE),
- Les conditions de validité juridique des documents électroniques.

Elle vise à sécuriser les transactions numériques entre citoyens, entreprises et administrations.

- **Loi n° 53-05 : Échange électronique de données juridiques**

Cette loi est l'une des premières à avoir posé les bases de la dématérialisation au Maroc. Elle reconnaît la validité juridique des documents électroniques, encadre leur conservation et définit les mécanismes permettant de garantir leur intégrité et authenticité.

- **Loi n° 0G-08 : Protection des données à caractère personnel**

Cette loi, promulguée en 2009, institue un cadre légal pour la protection des données personnelles des citoyens. Elle introduit :

- L'obligation d'obtenir le consentement explicite pour le traitement des données,
- Le droit d'accès, de rectification et d'opposition des individus,
- La création de la CNDP (Commission Nationale de contrôle de la protection des Données à caractère Personnel) chargée de veiller à l'application de la loi.

- **Loi n° 31-08 : Protection du consommateur**

Bien qu'elle ne soit pas dédiée exclusivement à la cybersécurité, cette loi intègre des dispositions sur la protection des consommateurs dans le cadre des transactions électroniques. Elle impose notamment des obligations de transparence, de sécurité des paiements et d'information dans les services en ligne.

- **Loi n° 24-G6 : Poste et télécommunications**

Cette loi encadre les services postaux et de télécommunications au Maroc. Elle contient des dispositions relatives :

- À la confidentialité des communications électroniques,
- À l'attribution des ressources (adresses IP, fréquences),
- À la régulation des opérateurs dans le respect de la sécurité des réseaux.

- **Loi n° 07-03 : Infractions relatives aux systèmes informatiques**

Adoptée en 2003, cette loi pénalise les infractions liées aux systèmes informatiques, notamment :

- L'accès frauduleux à un système,
- L'introduction, modification ou suppression non autorisée de données,
- L'usage de logiciels malveillants.

Elle constitue le socle du droit pénal informatique marocain et permet de lutter contre la cybercriminalité.

Missions et Tâches Réalisées

Tâche 1 : Implémentation d'un IDS/IPS avec Suricata et Wazuh

Dans cette tâche, j'ai conçu et mis en œuvre une architecture de sécurité basée sur Suricata (IDS/IPS) et Wazuh (SIEM). Les étapes réalisées incluent :

- Installation et configuration de Suricata sur une machine dédiée pour surveiller le trafic réseau et générer des alertes en fonction des règles prédéfinies (Snort/Suricata).
- Création de règles personnalisées pour détecter certains comportements malveillants (scan, XSS, RFI, etc.).
- Déploiement de Wazuh Manager et Wazuh Agent, permettant de centraliser et corrélérer les logs de sécurité.
- Intégration des logs Suricata dans Wazuh sans passer par syslog, en configurant une liaison directe pour optimiser la visibilité.
- Visualisation des alertes via le tableau de bord Wazuh et analyse des événements détectés.
- Déploiement d'un honeypot (DVWA) afin de générer des attaques réalistes et tester l'efficacité du système de détection.

Tâche 2 : Mise en place de contrôles de sécurité essentiels sur un poste utilisateur

Cette mission s'est concentrée sur le renforcement de la sécurité d'un poste Windows, souvent cible d'attaques internes ou externes. Les étapes clés ont été :

- Vérification de la version de Windows installée et évaluation de sa compatibilité avec Windows 11, dans le but d'anticiper la fin du support de Windows 10.
- Installation et configuration de l'antivirus Kaspersky, avec activation des fonctions de détection comportementale, surveillance réseau et pare-feu.
- Analyse des connexions applicatives via My Kaspersky, pour identifier les activités anormales.
- Apprentissage des fonctionnalités pare-feu de l'antivirus : blocage d'IP, filtrage de protocoles, création de règles personnalisées.

Contexte général

I. Introduction

Ce chapitre présente le contexte général du projet envisagé, incluant la présentation de l'organisme d'accueil : l'entreprise , l'établissement et la filière, la problématique à laquelle notre projet cherche à répondre, ainsi que les solutions proposées et qui se présentent dans les objectifs du projet.

II. Présentation de l'entreprise

1. Agence du Bassin Hydraulique du Tensift (ABHT)

L'Agence du Bassin Hydraulique du Tensift (ABHT) est un établissement public marocain instauré par le décret n° 2-00-479 du 14 novembre 2000, conformément à l'article 20 de la loi sur l'eau. Elle joue un rôle stratégique dans la gestion et la préservation des ressources hydriques au niveau du bassin du Tensift, qui englobe plusieurs régions hydrauliquement sensibles.

Mission principale :

ABHT agit en tant qu'acteur clé de la gestion intégrée et concertée de l'eau. Elle assure l'équilibre entre les besoins croissants en eau (agriculture, industrie, consommation domestique) et la préservation de cette ressource vitale, dans une logique de développement durable.

Vision de l'agence :

« Assurer une gestion intégrée et concertée des ressources en eau pour un développement durable du Bassin Hydraulique du Tensift. »



➤ Figure 1: logo de ABHT

Pour plus d'informations, leur site officiel : <https://abht.ma>

1.2 Le département informatique : un acteur clé de la transformation numérique

Dans le cadre de la transformation numérique de l'administration publique, le département informatique de l'ABHT joue un rôle fondamental. Il assure la conception, le déploiement, la sécurisation et la maintenance des services numériques internes et externes de l'agence.

Missions principales du service informatique :

- Gestion du parc informatique et des réseaux de l'agence.
- Administration des serveurs, bases de données et systèmes métiers.
- Déploiement et supervision des applications web internes (gestion documentaire, SIG hydrique, e-services...).
- Sécurisation des données et des communications.

- Mise en œuvre de politiques de cybersécurité et sensibilisation des utilisateurs.

1.3 Contexte de mon stage

Au sein du département informatique de l'ABHT, j'ai eu l'opportunité de participer à un projet concret de renforcement de la sécurité informatique, en mettant en œuvre un système de détection d'intrusion (IDS) basé sur les technologies Suricata, Wazuh et VirusTotal.

Ce projet s'inscrit dans une démarche proactive de l'agence pour :

- Prévenir les menaces informatiques potentielles.
- Surveiller le réseau interne contre les comportements suspects.
- Automatiser la réponse aux incidents grâce à des scripts personnalisés.
- Protéger les données sensibles liées à la gestion de l'eau et aux infrastructures hydrauliques.

Ce stage m'a permis d'appliquer mes connaissances en cybersécurité dans un environnement professionnel réel, tout en contribuant à un projet à fort impact pour l'agence.

2. ENSA :

L'École Nationale des Sciences Appliquées (ENSA) de Marrakech est fondée en 2000, elle a pour mission de former les futurs cadres ingénieurs à travers une pédagogie active, favorisant l'ouverture professionnelle, internationale et culturelle.

Affiliée à l'UCA, l'ENSA de Marrakech joue un rôle de leader dans les filières d'ingénierie. Son impact est significatif, garantissant une formation de qualité et un développement de compétences exceptionnel. L'école favorise un climat de confiance basé sur un engagement mutuel entre les élèves ingénieurs, les enseignants, et le personnel administratif et technique.

Elle comporte 5 filières : Génie Informatique, Génie Industriel et Logistique, Génie Réseau, systèmes et Service programmable, Génie Systèmes Electroniques Embarqués et Commande des Systèmes, et Génie Cyber Défense et Systèmes de Télécommunication Embarqués.



➤ Figure 2: logo de ENSA

3. GCDSTE

La filière Génie Cyberdéfense et Système de Télécommunications Embarquée (GCDSTE) de l'ENSA-M a pour objectif de former des ingénieurs compétents dotés de solides connaissances scientifiques.

Cette formation intègre des méthodes et des techniques avancées pour assurer la sécurité et gérer les risques liés aux systèmes d'information. De

plus, elle vise à habilitier les étudiants à concevoir des systèmes de télécommunications embarqués.

L'accent est mis sur la préparation de chaque élève ingénieur aux méthodes et outils nécessaires pour contrer la cybercriminalité, résoudre les failles des systèmes d'information, et aborder les défis de la sécurité numérique.



➤ Figure 3: logo de GCDSTE

3.1 Contexte et objectifs du projet

Ce projet a été conçu dans l'optique d'étudier et d'expérimenter la mise en œuvre d'un système de détection d'intrusion moderne, alliant les performances de Suricata et la puissance d'analyse de Wazuh. L'objectif principal est de disposer d'une plateforme complète capable de :

- Surveiller en continu le trafic réseau afin d'identifier tout signe d'activité malveillante ;

- Collecter et corrélérer les événements de sécurité provenant des différentes sources (logs système, journaux applicatifs, alertes IDS) ;
- Alerter les administrateurs de manière réactive et documenter les incidents pour faciliter l'analyse et la réponse.

En plus de son aspect technique, cette démarche a également une visée pédagogique : elle permet de renforcer la compréhension des concepts de détection d'intrusion, de gestion des alertes et de mise en place d'un environnement sécurisé et supervisé.

3.2 Importance des systèmes de détection d'intrusion (IDS)

Les systèmes de détection d'intrusion jouent un rôle crucial dans l'arsenal de sécurité d'une organisation. Leur fonction consiste à analyser le trafic réseau ou les activités système dans le but de détecter des comportements anormaux, susceptibles de révéler une tentative d'attaque ou une compromission.

Un IDS comme Suricata est capable d'inspecter les paquets en profondeur (Deep Packet Inspection), de repérer les signatures connues d'attaques et de générer des alertes en temps réel. Lorsqu'il est associé à une plateforme centralisée telle que Wazuh, il devient possible de corrélérer les événements, de générer des rapports détaillés et d'orchestrer la réponse aux incidents de façon structurée.

Dans un contexte où les cyberattaques sont de plus en plus sophistiquées et fréquentes, l'IDS constitue un rempart essentiel qui complète les mesures préventives (pare-feu, antivirus, politiques de sécurité) et contribue à réduire significativement le risque de compromission.

Étude des Outils Utilisés

I. Suricata : IDS/IPS performant



➤ Figure 4: logo de SURICATA

1. Qu'est-ce que Suricata ?

Suricata est un logiciel libre de détection et de prévention d'intrusion (IDS/IPS), ainsi qu'un outil de surveillance réseau (NSM – Network Security Monitoring). Il permet d'analyser le trafic réseau en temps réel afin de détecter des attaques ou comportements suspects.

2. Caractéristiques :

- Détection d'intrusions : Utilise des règles (similaires à Snort) pour identifier les signatures d'attaques connues.
- Inspection approfondie des paquets (DPI) : Capable d'analyser les protocoles comme HTTP, TLS, DNS, FTP, etc.
- Multithreading : Suricata peut utiliser plusieurs cœurs du processeur, ce qui le rend performant sur des réseaux à fort trafic.
- Journalisation détaillée : Génère différents types de logs, notamment des fichiers JSON, très utiles pour l'analyse des alertes.
- Extraction de fichiers : Permet d'extraire automatiquement certains fichiers transmis sur le réseau

II. Honeypot DVWA



➤ Figure 5: logo de DVWA

1. Qu'est-ce que DVWA ?

Dans le cadre de ce projet IDS, un honeypot DVWA (Damn Vulnerable Web Application) a été déployé afin de simuler une application web vulnérable accessible depuis le réseau. L'objectif est d'attirer d'éventuelles attaques (comme les injections SQL, XSS, brute force, etc.) et d'observer comment ces attaques sont détectées et journalisées par les outils de détection mis en place.

DVWA constitue un environnement contrôlé permettant de générer du trafic malveillant légitime. Ce trafic est ensuite analysé par Suricata, qui inspecte les paquets en profondeur et déclenche des alertes basées sur des signatures prédéfinies. Les journaux générés par Suricata sont ensuite centralisés et corrélés via Wazuh, qui joue ici un rôle de SIEM léger, facilitant la visualisation, l'analyse et l'alerte en temps réel.

2. Caractéristiques :

- De valider l'efficacité du pipeline de détection.
- De mieux comprendre les mécanismes d'attaque web.
- D'évaluer la pertinence des règles Suricata dans un contexte réaliste.

III. Wazuh : Plateforme SIEM pour surveillance et détection d'anomalies :



➤ Figure 6: logo de Wazuh

1. Qu'est-ce que Wazuh ?

Wazuh est une plateforme open-source de type SIEM (Security Information and Event Management) qui permet de collecter, analyser, corréliser et visualiser des événements de sécurité provenant de multiples sources (agents, journaux, systèmes IDS, etc.).

2. Caractéristiques principales :

- Détection d'intrusions (HIDS) : surveille l'intégrité des fichiers, les journaux, les rootkits, etc.
- Collecte et centralisation des logs via l'agent Wazuh.
- Analyse comportementale et détection d'anomalies.
- Alertes en temps réel basées sur des règles configurables.
- Tableau de bord visuel grâce à l'intégration avec Kibana/Elastic Stack.
- Gestion des vulnérabilités, conformité (PCI DSS, GDPR, etc.).
- Compatible avec Suricata, Zeek, osquery, Docker, Kubernetes, etc.
- Extensible et personnalisable selon les besoins du système

IV. AbuseIPDB : Base de données d'adresses IP malveillantes



➤ Figure 7: logo de AbuseIBDB

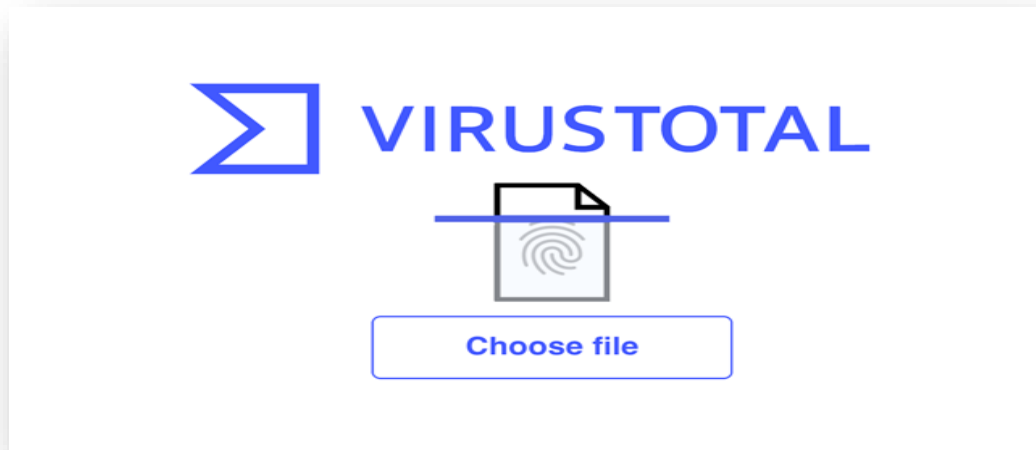
1. Qu'est-ce qu'AbuseIPDB ?

AbuseIPDB (Abuse IP Database) est une plateforme collaborative conçue pour signaler et consulter des adresses IP suspectes. Elle collecte des informations provenant de nombreux utilisateurs et systèmes dans le monde entier, concernant des comportements malveillants tels que le spam, les tentatives de connexion brute force, les scans de ports ou encore les attaques DDoS. Chaque adresse IP répertoriée dispose d'un score de confiance, mis à jour selon la fréquence et la gravité des signalements. Grâce à une API publique, les développeurs et les administrateurs peuvent intégrer cette base dans leurs outils de surveillance ou de filtrage. Cela permet une meilleure réactivité face aux menaces et favorise une cybersécurité plus collaborative.

2. Caractéristiques principales :

- Score de réputation basé sur des signalements globaux.
- API RESTful simple à utiliser avec Python ou Bash.
- Historique des signalements disponible pour chaque IP.
- Possibilité de filtrer par type de menace (SSH, spam, etc.).
- Base régulièrement mise à jour par la communauté.

V. VirusTotal :API



➤ Figure 8: logo de VirusTotal

1. Qu'est-ce que VirusTotal ?

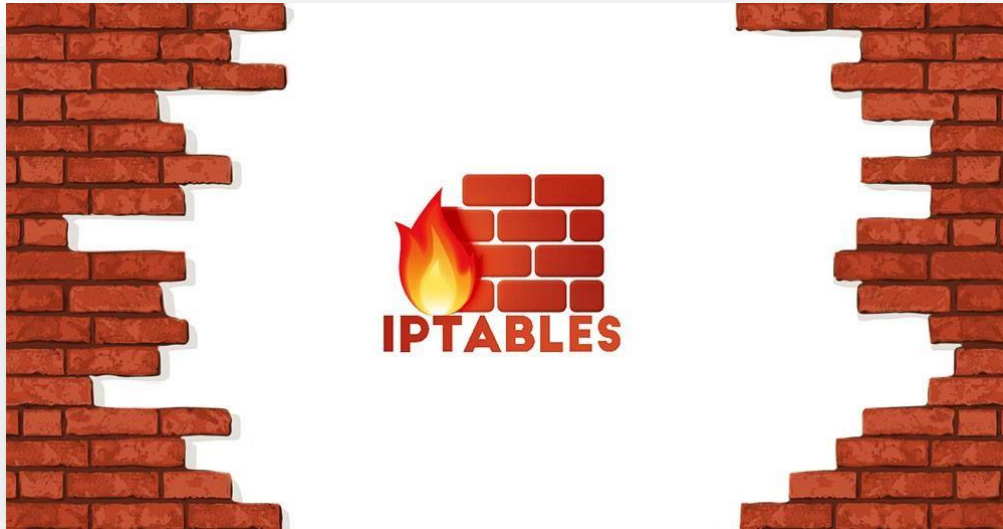
VirusTotal est un service en ligne gratuit qui analyse des fichiers, des URLs, des adresses IP et des domaines suspects afin de détecter des logiciels malveillants, chevaux de Troie, vers, ou autres types de menaces. Il s'appuie sur les moteurs antivirus de plusieurs éditeurs renommés ainsi que sur des outils d'analyse heuristique et comportementale. VirusTotal permet également la recherche dans des bases de données de menaces connues, ce qui en fait un outil précieux pour les analystes en cybersécurité.

Dans le cadre de Wazuh, l'API VirusTotal peut être utilisée pour enrichir les alertes générées par les agents, en consultant automatiquement la réputation d'un fichier hashé (SHA256/MD5) ou d'une adresse IP/domaines observés. Cela offre un contexte supplémentaire lors de l'analyse d'incidents de sécurité.

2. Caractéristiques principales :

- Analyse multi-antivirus (plus de 70 moteurs de détection).
- API RESTful permettant l'automatisation des recherches (fichiers, IP, URLs, domaines).
- Informations détaillées : score de détection, noms de menaces, comportements observés.

VI. iptables : Le pare-feu natif de Linux



➤ Figure 9: logo de IPTABLES

1. Qu'est-ce que iptables ?

Iptables est un outil de gestion de pare-feu utilisé dans les systèmes Linux pour contrôler le trafic réseau. Il fonctionne grâce au système Netfilter, intégré au noyau Linux, permettant de créer des règles précises pour filtrer les paquets entrants, sortants ou transférés. Ces règles peuvent être appliquées en fonction de différents critères comme l'adresse IP source ou destination, le port utilisé, ou encore le protocole. Très souple et puissant, iptables est utilisé dans de nombreux environnements, allant des serveurs personnels aux grandes infrastructures d'entreprise. Il permet non seulement de sécuriser un système contre des accès non autorisés, mais aussi de limiter le trafic selon des politiques précises.

2. Caractéristiques principales :

- Fonctionne en ligne de commande avec des règles dynamiques
- Permet de filtrer les paquets par IP, port ou protocole
- Possibilité de journaliser les connexions bloquées
- Supporte des règles persistantes via des fichiers de configuration
- Intégré nativement dans toutes les distributions Linux

VII. Environnement de virtualisation

Dans le cadre de ce projet, l'outil VMware Workstation a été utilisé pour virtualiser l'environnement de cybersécurité, facilitant ainsi le déploiement, la gestion et l'expérimentation de l'architecture. Deux machines virtuelles ont été mises en place :

- La première hébergeait le honeypot ainsi que le système de détection d'intrusion Suricata.
- La seconde était dédiée au serveur Wazuh, chargé de centraliser, analyser et corréler les alertes générées.

Cette séparation des rôles permet de reproduire un environnement réaliste et sécurisé, tout en assurant une grande flexibilité pour les tests. Grâce à la virtualisation, les manipulations peuvent être effectuées sans risque pour la machine physique, dans un réseau isolé. De plus, VMware offre une gestion fine des ressources et la possibilité de créer des snapshots, ce qui permet de restaurer rapidement un état antérieur en cas d'erreur ou pour répéter certains tests.

1. Présentation de VMware Workstation

VMware Workstation est un logiciel de virtualisation permettant de créer et d'exécuter plusieurs machines virtuelles sur un seul ordinateur physique. Chaque machine virtuelle fonctionne de manière indépendante avec son propre système d'exploitation.



➤ Figure 10: logo de VMware

2. Principales caractéristiques :

- Exécution simultanée de plusieurs systèmes d'exploitation : Il est possible de faire fonctionner Windows, Linux, ou d'autres OS en parallèle sur différentes VM.
- Support des systèmes 64 bits et des environnements de test : VMware permet la création de laboratoires de développement, de tests réseau ou de machines dédiées à des usages spécifiques.
- Snapshots : La possibilité de capturer l'état d'une machine virtuelle à un instant donné permet de revenir facilement à cette configuration, ce qui est très utile lors des phases de test ou de débogage.

3. Cas d'usage :

VMware Workstation est principalement utilisé pour :

- Tester des systèmes d'exploitation et des configurations réseau dans un environnement isolé.
- Développer et tester des applications sans impacter le système hôte.
- Simuler des scénarios d'attaque ou de défense en cybersécurité.

Il est largement adopté par les développeurs, les administrateurs système ainsi que les professionnels de la sécurité informatique pour sa robustesse et sa souplesse d'utilisation.

Implémentation et Réalisation

I. Introduction :

Après avoir présenté les outils et technologies sélectionnés pour notre projet de cybersécurité dans le chapitre précédent, ce chapitre s'attache à décrire la mise en œuvre concrète de notre infrastructure de détection d'intrusion. L'objectif principal est de montrer comment les différents composants ont été installés, configurés et intégrés dans un environnement de test virtuel, conçu pour simuler des attaques réelles et en analyser les alertes.

Le cœur de notre projet repose sur trois éléments clés :

- Le système de détection d'intrusion **Suricata**, chargé de surveiller le trafic réseau en temps réel.
- La solution de supervision **Wazuh**, utilisée pour centraliser, corrélérer et visualiser les alertes générées par Suricata et d'autres sources.
- Un **honeypot** basé sur l'application vulnérable **DVWA (Damn Vulnerable Web Application)**, servant de cible pour les attaques simulées.

L'ensemble a été déployé dans un environnement virtualisé via VMware Workstation, offrant un cadre isolé, flexible et sécurisé pour les expérimentations. Ce chapitre présente en détail les différentes étapes du déploiement, depuis l'installation des machines virtuelles jusqu'à la configuration fine des règles Suricata et des agents Wazuh.

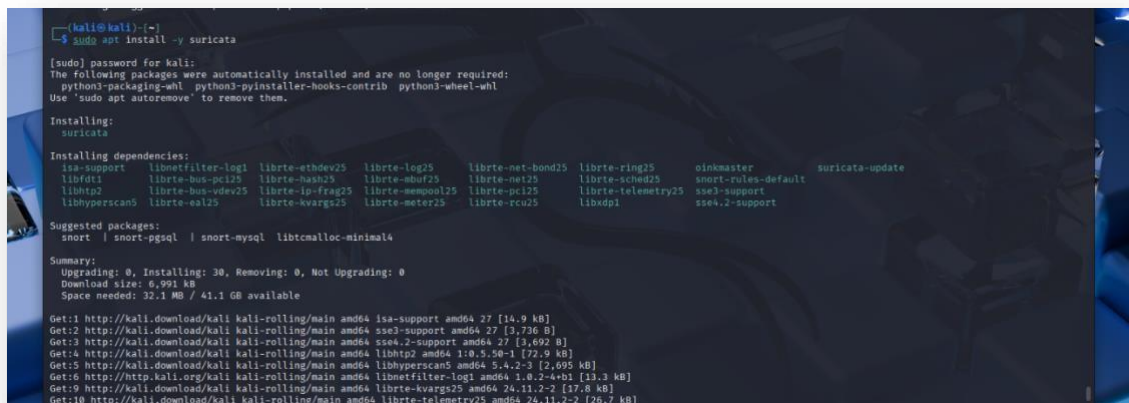
Nous reviendrons également sur les défis techniques rencontrés, notamment en ce qui concerne la communication entre les composants, la configuration des logs et la génération d'alertes pertinentes. Des solutions concrètes seront apportées pour chaque problème identifié.

Enfin, nous procéderons à des tests d'intrusion sur le honeypot DVWA afin d'observer en temps réel le comportement du système de détection et de mesurer l'efficacité globale de notre architecture. Ce travail mettra en lumière la transition entre théorie et pratique, illustrant ainsi la capacité à mettre en place une solution IDS/IPS réaliste et fonctionnelle dans un environnement maîtrisé.

II. Installation et configuration de Suricata :

Dans cette partie, nous allons détailler les étapes suivies pour installer et configurer Suricata sur notre machine dédiée à la surveillance du trafic réseau. Suricata jouera ici le rôle principal du système de détection d'intrusion (IDS), analysant les paquets réseau échangés entre les différentes machines de notre environnement

L'installation a été réalisée via les dépôts officiels avec les commandes suivantes :



```
(kali@kali)~$ sudo apt install -y suricata
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  python3-packaging-whl python3-pyinstaller-hooks-contrib python3-wheel-whl
Use 'sudo apt autoremove' to remove them.

Installing:
  suricata

Installing dependencies:
  isa-support  libnetfilter-logmsg  librtre-ethdev25  librtre-log25  librtre-net-bond25  librtre-ring25  oinkmaster  suricata-update
  librtre1    librtre-bus-pci25  librtre-nash25   librtre-mbuf25  librtre-net25      librtre-sched25  snort-rules-default
  librtre2    librtre-bus-vdev25  librtre-ip-frag25  librtre-mempool25  librtre-pci25      librtre-telemetry25  sse4.2-support
  libhyperscan5  librtre-eal25      librtre-kvargs25  librtre-meter25  librtre-rcu25      libxdpi

Suggested packages:
  snort | snort-pgsql | snort-mysql  libtcmalloc-minimal4

Summary:
  Upgrading: 0, Installing: 30, Removing: 0, Not Upgrading: 0
  Download size: 6,991 kB
  Space needed: 32.1 MB / 41.1 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 isa-support amd64 27 [14.9 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 sse4.2-support amd64 27 [3,736 B]
Get:3 http://kali.download/kali kali-rolling/main amd64 sse4.2-support amd64 27 [3,692 B]
Get:4 http://kali.download/kali kali-rolling/main amd64 librtre1 amd64 1:0.5.50-1 [72.9 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 libhyperscan5 amd64 5.4.2-3 [2,695 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 libnetfilter-logmsg amd64 1.0.2-4+b1 [13.3 kB]
Get:7 http://kali.download/kali kali-rolling/main amd64 librtre-kvargs25 amd64 24.11.2-2 [17.8 kB]
Get:8 http://kali.download/kali kali-rolling/main amd64 librtre-telemetry25 amd64 24.11.2-2 [26.7 kB]
```

➤ Figure 11: Installation suricata

Une fois installé, nous avons vérifié la version pour s'assurer de la bonne installation :

- `suricata --build-info`

1. Configuration de base

La configuration principale de Suricata se trouve dans le fichier suivant :

`/etc/suricata/suricata.yaml`

Nous avons modifié plusieurs sections importantes :

- **Interface réseau à surveiller**

Nous avons identifié l'interface à utiliser avec :

Puis défini cette interface dans le fichier YAML, section `af-packet` :

af-packet:

- interface: eth0

Remarque : l'interface peut aussi être ens33, eth1, etc. selon l'environnement.

- **Configuration des logs**

Les logs ont été configurés pour être envoyés à Wazuh via le système Syslog.

Nous avons activé les logs EVE (JSON) :

outputs:

- eve-log:

enabled: yes

filetype: regular

filename: /var/log/suricata/eve.json

```
GNU nano 8.4 /var/log/suricata/eve.json
timestamp "2025-07-06T12:21:01.262454+0100" event_type "stats" stats "uptime" 8 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 114 "poll_signal" 0 "poll_timeout" 114 "poll
timestamp "2025-07-06T12:21:09.265738+0100" event_type "stats" stats "uptime" 16 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 285 "poll_signal" 0 "poll_timeout" 285 "poll
timestamp "2025-07-06T12:21:17.269424+0100" event_type "stats" stats "uptime" 24 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 461 "poll_signal" 0 "poll_timeout" 461 "poll
timestamp "2025-07-06T12:21:25.270612+0100" event_type "stats" stats "uptime" 32 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 586 "poll_signal" 0 "poll_timeout" 586 "poll
timestamp "2025-07-06T12:21:33.272405+0100" event_type "stats" stats "uptime" 40 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 744 "poll_signal" 0 "poll_timeout" 744 "poll
timestamp "2025-07-06T12:21:41.274404+0100" event_type "stats" stats "uptime" 48 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 890 "poll_signal" 0 "poll_timeout" 890 "poll
timestamp "2025-07-06T12:21:49.275369+0100" event_type "stats" stats "uptime" 56 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1037 "poll_signal" 0 "poll_timeout" 1037 "poll
timestamp "2025-07-06T12:21:57.277176+0100" event_type "stats" stats "uptime" 64 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1216 "poll_signal" 0 "poll_timeout" 1216 "poll
timestamp "2025-07-06T12:22:05.291046+0100" event_type "stats" stats "uptime" 72 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1330 "poll_signal" 0 "poll_timeout" 1330 "poll
timestamp "2025-07-06T12:22:13.299662+0100" event_type "stats" stats "uptime" 80 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1488 "poll_signal" 0 "poll_timeout" 1488 "poll
timestamp "2025-07-06T12:22:21.461533+0100" event_type "stats" stats "uptime" 88 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1659 "poll_signal" 0 "poll_timeout" 1646 "poll
timestamp "2025-07-06T12:22:29.464123+0100" event_type "stats" stats "uptime" 96 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1829 "poll_signal" 0 "poll_timeout" 1816 "poll
timestamp "2025-07-06T12:22:37.465667+0100" event_type "stats" stats "uptime" 104 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1945 "poll_signal" 0 "poll_timeout" 1932 "poll
timestamp "2025-07-06T12:22:45.469336+0100" event_type "stats" stats "uptime" 112 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 2210 "poll_signal" 0 "poll_timeout" 2107 "poll
timestamp "2025-07-06T12:22:53.479909+0100" event_type "stats" stats "uptime" 120 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 2296 "poll_signal" 0 "poll_timeout" 2283 "poll
timestamp "2025-07-06T12:23:01.483412+0100" event_type "stats" stats "uptime" 128 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 2412 "poll_signal" 0 "poll_timeout" 2399 "poll
timestamp "2025-07-06T12:23:04.880810+0100" event_type "stats" stats "uptime" 131 "capture" "kernel_packets" 13 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 2509 "poll_signal" 0 "poll_timeout" 2494 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 4 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 43 "poll_signal" 0 "poll_timeout" 41 "poll da
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 8 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 117 "poll_signal" 0 "poll_timeout" 117 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 16 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 296 "poll_signal" 0 "poll_timeout" 296 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 24 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 415 "poll_signal" 0 "poll_timeout" 415 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 32 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 594 "poll_signal" 0 "poll_timeout" 594 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 40 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 773 "poll_signal" 0 "poll_timeout" 773 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 48 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 892 "poll_signal" 0 "poll_timeout" 892 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 56 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1071 "poll_signal" 0 "poll_timeout" 1071 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 64 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1249 "poll_signal" 0 "poll_timeout" 1249 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 72 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1434 "poll_signal" 0 "poll_timeout" 1434 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 80 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1614 "poll_signal" 0 "poll_timeout" 1614 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 88 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1792 "poll_signal" 0 "poll_timeout" 1792 "poll
timestamp "2025-07-06T12:24:48.271272+0100" event_type "stats" stats "uptime" 96 "capture" "kernel_packets" 0 "kernel_drops" 0 "errors" 0 "afpacket" "busy_loop_avg" 0 "polls" 1971 "poll_signal" 0 "poll_timeout" 1971 "poll
```

2. Gestion et configuration des règles Suricata

Suricata repose sur un système de règles pour détecter les menaces dans le trafic réseau. Ces règles définissent des signatures ou comportements suspects que Suricata doit surveiller.

a) Emplacement des règles

Les règles actives sont stockées dans le répertoire suivant :

```
/etc/suricata/rules/
```

Des règles personnalisées ont été ajoutées dans le fichier `suricata.yaml` pour adapter la détection aux besoins spécifiques du système surveillé. Cela permet d'élargir ou de restreindre la portée de détection de Suricata selon le contexte. Ce fichier `suricata.rules` contient l'ensemble des règles actives. Il est souvent mis à jour automatiquement avec `suricata-update`.

b) Téléchargement des règles communautaires (Emerging Threats)

Nous avons utilisé l'outil intégré `suricata-update` pour récupérer les dernières règles publiées par la communauté :

```
(kali㉿ kali) [~]
$ sudo suricata-update
/7/2025 -- 12:29:28 -- <Info> -- Using data-directory /var/lib/suricata.
/7/2025 -- 12:29:28 -- <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
/7/2025 -- 12:29:28 -- <Info> -- Using /etc/suricata/rules for Suricata provided rules.
/7/2025 -- 12:29:29 -- <Info> -- Found Suricata version 7.0.10 at /usr/bin/suricata.
/7/2025 -- 12:29:29 -- <Info> -- Loading /etc/suricata/suricata.yaml
/7/2025 -- 12:29:29 -- <Info> -- Disabling rules for protocol pgsq
/7/2025 -- 12:29:29 -- <Info> -- Disabling rules for protocol modbus
/7/2025 -- 12:29:29 -- <Info> -- Disabling rules for protocol dnp3
/7/2025 -- 12:29:29 -- <Info> -- Disabling rules for protocol enip
/7/2025 -- 12:29:29 -- <Info> -- No sources configured, will use Emerging Threats Open
/7/2025 -- 12:29:29 -- <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-7.0.10/emerging.rules.tar.gz.
```

➤ Figure 13: Téléchargement des règles

Cela télécharge les règles depuis Emerging Threats (ET Open), une source reconnue pour sa base de signatures open source.

c) Tester les règles

Après modification ou ajout de règles, il est obligatoire de tester la configuration :

```
(kali㉿ kali) [~] wazuh-syscheckd: WARNING: Process locked due to agent is offline. Waiting
$ suricata -T -c /etc/suricata/suricata.yaml -S /var/lib/suricata/rules/my_custom.rules
2025/04/08 23:15:09 wazuh-agentd: INFO: Trying to connect to server ([172.17.119.120]:1514/tcp).
2025/04/08 23:16:20 wazuh-agentd: INFO: Closing connection to server ([172.17.119.120]:1514/tcp).
Info: conf-yaml-loader: Configuration node 'types' redefined.
Info: conf-yaml-loader: Configuration node 'stream' redefined.
i: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode.
i: suricata: Configuration provided was successfully loaded. Exiting. Process unlocked, continuing
```

➤ Figure 14: Tester les règles télécharger

Le test valide le fichier YAML et les règles chargées.

Ensuite, nous avons redémarré Suricata :

```
(kali㉿ kali) [~] wazuh-modulesd:syscollector: INFO: Evaluation finished.
$ sudo systemctl status suricata
2025/04/08 23:10:26 wazuh-agentd: INFO: Closing connection to server ([172.17.119.120]:1514/tcp).
[sudo] Mot de passe de kali :
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
   Active: active (running) since Wed 2025-04-09 15:42:53 +01; 1 day 20h ago
```

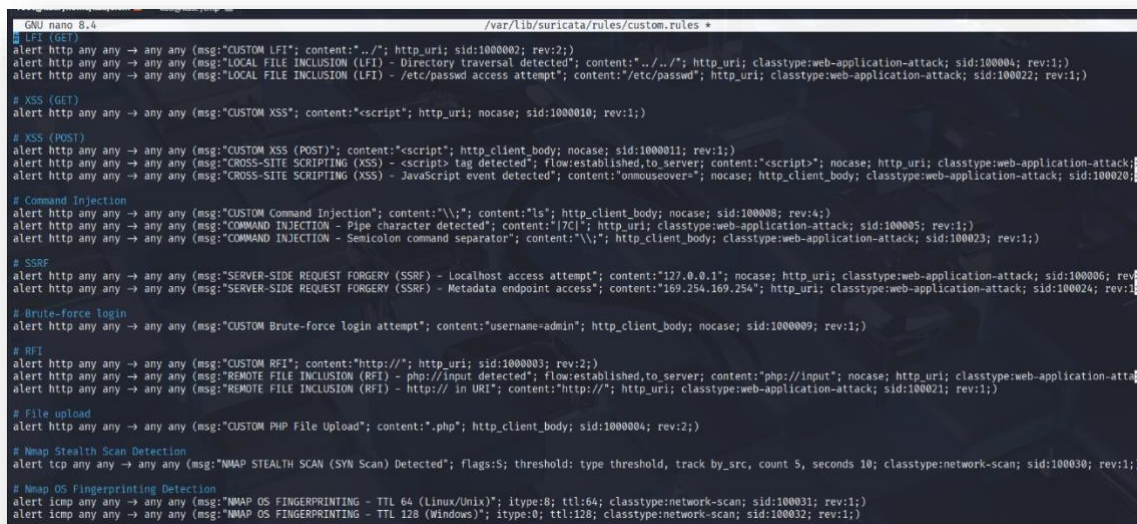
➤ Figure 14: redémarré Suricata

d) Création de règles personnalisées

Pour tester notre système, nous avons également rédigé des règles personnalisées simples, notamment pour détecter des requêtes spécifiques sur le honeypot DVWA :

Ces règles ont été ajoutées dans un fichier séparé, par exemple local.rules, puis référencées dans suricata.yaml :

- rule-files:
 - local.rules
 - suricata.rules



```
GNU nano 8.4 /var/lib/suricata/rules/custom.rules
# LFI (GET)
alert http any any -> any any (msg:"CUSTOM LFI"; content:"../"; http_uri; sid:1000002; rev:2;)
alert http any any -> any any (msg:"LOCAL FILE INCLUSION (LFI) - Directory traversal detected"; content:"../"; http_uri; classtype:web-application-attack; sid:1000004; rev:1;)
alert http any any -> any any (msg:"LOCAL FILE INCLUSION (LFI) - /etc/passwd access attempt"; content:"/etc/passwd"; http_uri; classtype:web-application-attack; sid:100022; rev:1;)

# XSS (GET)
alert http any any -> any any (msg:"CUSTOM XSS"; content:"<script>"; http_uri; nocase; sid:1000010; rev:1;)

# XSS (POST)
alert http any any -> any any (msg:"CUSTOM XSS (POST)"; content:"<script>"; http_client_body; nocase; sid:1000011; rev:1;)
alert http any any -> any any (msg:"CROSS-SITE SCRIPTING (XSS) - <script> tag detected"; flow:established,to_server; content:"<script>"; nocase; http_uri; classtype:web-application-attack; sid:1000012; rev:1;)
alert http any any -> any any (msg:"CROSS-SITE SCRIPTING (XSS) - JavaScript event detected"; content:"onmouseover="; nocase; http_client_body; classtype:web-application-attack; sid:100020; rev:1;)

# Command Injection
alert http any any -> any any (msg:"CUSTOM Command Injection"; content:"\\\\"; content:"ls"; http_client_body; nocase; sid:1000008; rev:4;)
alert http any any -> any any (msg:"COMMAND INJECTION - Pipe character detected"; content:"|7C|"; http_uri; classtype:web-application-attack; sid:100005; rev:1;)
alert http any any -> any any (msg:"COMMAND INJECTION - Semicolon command separator"; content:"\\\\"; http_client_body; classtype:web-application-attack; sid:100023; rev:1;)

# SSRF
alert http any any -> any any (msg:"SERVER-SIDE REQUEST FORGERY (SSRF) - Localhost access attempt"; content:"127.0.0.1"; nocase; http_uri; classtype:web-application-attack; sid:100006; rev:1;)
alert http any any -> any any (msg:"SERVER-SIDE REQUEST FORGERY (SSRF) - Metadata endpoint access"; content:"169.254.169.254"; http_uri; classtype:web-application-attack; sid:100024; rev:1;)

# Brute-force login
alert http any any -> any any (msg:"CUSTOM Brute-force login attempt"; content:"username=admin"; http_client_body; nocase; sid:1000009; rev:1;)

# RFI
alert http any any -> any any (msg:"CUSTOM RFI"; content:"http://"; http_uri; sid:1000003; rev:2;)
alert http any any -> any any (msg:"REMOTE FILE INCLUSION (RFI) - php://input detected"; flow:established,to_server; content:"php://input"; nocase; http_uri; classtype:web-application-attack; sid:100021; rev:1;)
alert http any any -> any any (msg:"REMOTE FILE INCLUSION (RFI) - http:// in URI"; content:"http://"; http_uri; classtype:web-application-attack; sid:100021; rev:1;)

# File upload
alert http any any -> any any (msg:"CUSTOM PHP File Upload"; content:".php"; http_client_body; sid:1000004; rev:2;)

# Nmap Stealth Scan Detection
alert tcp any any -> any any (msg:"NMAP STEALTH SCAN (SYN Scan) Detected"; flags:S; threshold: type threshold, track by_src, count 5, seconds 10; classtype:network-scan; sid:100030; rev:1;)

# Nmap OS Fingerprinting Detection
alert icmp any any -> any any (msg:"NMAP OS FINGERPRINTING - TTL 64 (Linux/Unix)"; itype:8; ttl:64; classtype:network-scan; sid:100031; rev:1;)
alert icmp any any -> any any (msg:"NMAP OS FINGERPRINTING - TTL 128 (Windows)"; itype:0; ttl:128; classtype:network-scan; sid:100032; rev:1;)
```

➤ Figure 15: règles personnalisées

3. Résultats de l'analyse :

Une fois Suricata lancé et les règles personnalisées activées, nous avons simulé plusieurs types d'attaques sur notre site web vulnérable (honeypot). Ces attaques incluait des tentatives de :

- Cross-Site Scripting (XSS)
- Injection SQL
- Brute force
- RFI et LFI
- Ssrf

```

04/11/2025-12:47:29.244142 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51432 -> 172.17.125.41:5000
04/11/2025-12:47:29.264052 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51433 -> 172.17.125.41:5000
04/11/2025-12:47:29.281925 [**] [1:1000062:3] XSS Attempt - Script Tag Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51434 -> 172.17.125.41:5000
04/11/2025-12:47:29.281925 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51434 -> 172.17.125.41:5000
04/11/2025-12:47:29.307977 [**] [1:1000053:1] Logical Flaw - Negative Amount Detected [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 172.17.112.1:51436 -> 172.17.125.41:5000
04/11/2025-12:47:29.330039 [**] [1:1000074:1] Logical Flaw - Excessive Amount Detected [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 172.17.112.1:51437 -> 172.17.125.41:5000
04/11/2025-12:48:00.297635 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51793 -> 172.17.125.41:5000
04/11/2025-12:48:00.374671 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51797 -> 172.17.125.41:5000
04/11/2025-12:48:00.468905 [**] [1:1000101:1] XSS in Transfers Description [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51799 -> 172.17.125.41:5000
04/11/2025-12:48:00.468905 [**] [1:1000111:1] SQL Injection Attempt Detected [**] [Classification: Web Application Attack] [Priority: 1] {TCP} 172.17.112.1:51799 -> 172.17.125.41:5000
04/11/2025-12:48:00.503464 [**] [1:1000102:1] Negative Transfer Amount [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 172.17.112.1:51802 -> 172.17.125.41:5000
04/11/2025-12:48:00.534624 [**] [1:1000103:1] Suspicious Large Transfer [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 172.17.112.1:51805 -> 172.17.125.41:5000
04/11/2025-12:48:04.530775 [**] [1:2221043:1] SURICATA HTTP Request double encoded URI [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 172.17.112.1:51855 -> 172.17.125.41:5000
04/11/2025-12:48:04.545146 [**] [1:2221043:1] SURICATA HTTP Request double encoded URI [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {TCP} 172.17.112.1:51856 -> 172.17.125.41:5000

```

➤ Figure 16: fichier de log fast.log (Suricata)

Résultats observés

Les résultats ont été collectés à partir du fichier de log fast.log, généré automatiquement par Suricata. Voici un résumé des détections effectuées :

- **SQL Injection** : Tentatives directes d'injections SQL visant à manipuler des requêtes

en base de données.

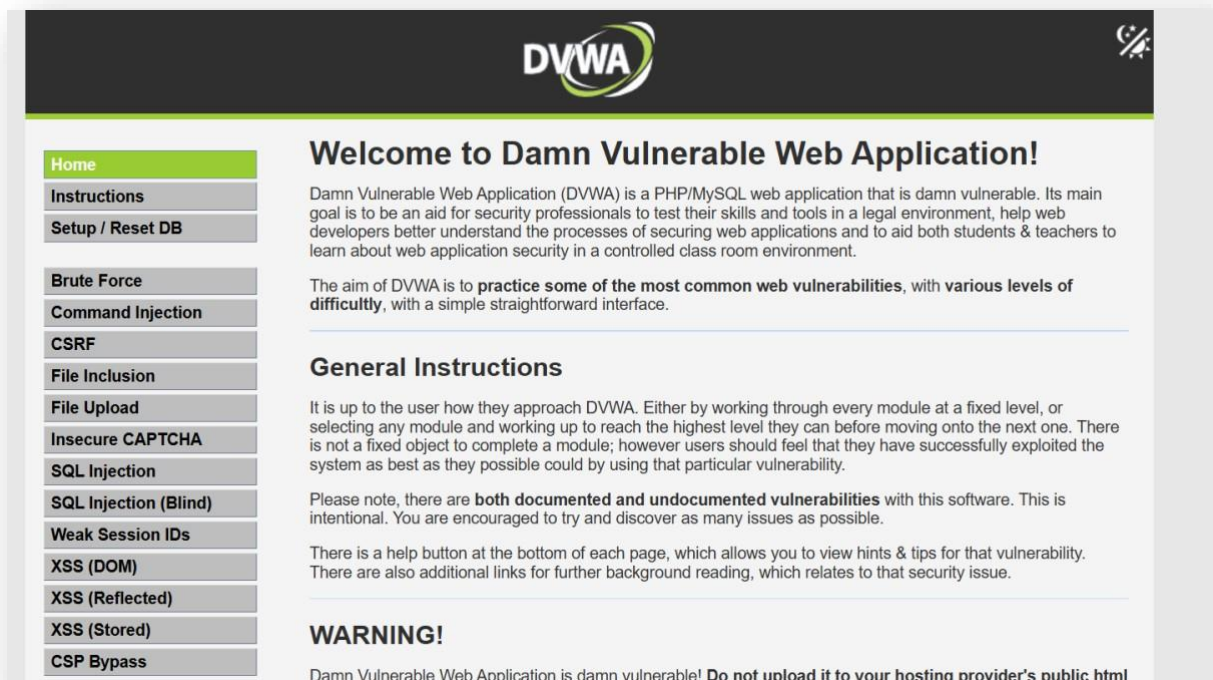
- **Cross-Site Scripting (XSS)** : Injections de scripts dans les champs texte ou description pour exécuter du JavaScript malveillant.

III. Déploiement du honeypot DVWA pour la détection d'attaques

1. Qu'est-ce que DVWA ?

DVWA (Damn Vulnerable Web Application) est une application web volontairement vulnérable, conçue pour permettre aux chercheurs en sécurité, aux pentesters et aux étudiants de tester diverses failles applicatives dans un environnement contrôlé.

Dans notre projet IDS, nous avons déployé DVWA sur une machine virtuelle Linux pour jouer le rôle de **honeypot**. Le but était d'attirer des attaques afin d'évaluer la capacité de détection de notre système Wazuh + Suricata + VirusTotal.



➤ Figure 17:DVWA interface

2. Configuration du honeypot

1. Installation d'un serveur LAMP (Linux, Apache, MySQL, PHP).
2. Clonage du projet DVWA depuis GitHub dans le répertoire `/var/www/html/dvwa`.
3. Configuration de la base de données dans le fichier `config.inc.php`.
4. Ouverture volontaire de ports (HTTP - 80, SSH - 22) dans la VM pour rendre la machine "appétissante" aux attaquants.
5. L'interface réseau de la machine DVWA a été bridgée ou placée sur le même réseau que Suricata pour capter le trafic.

```

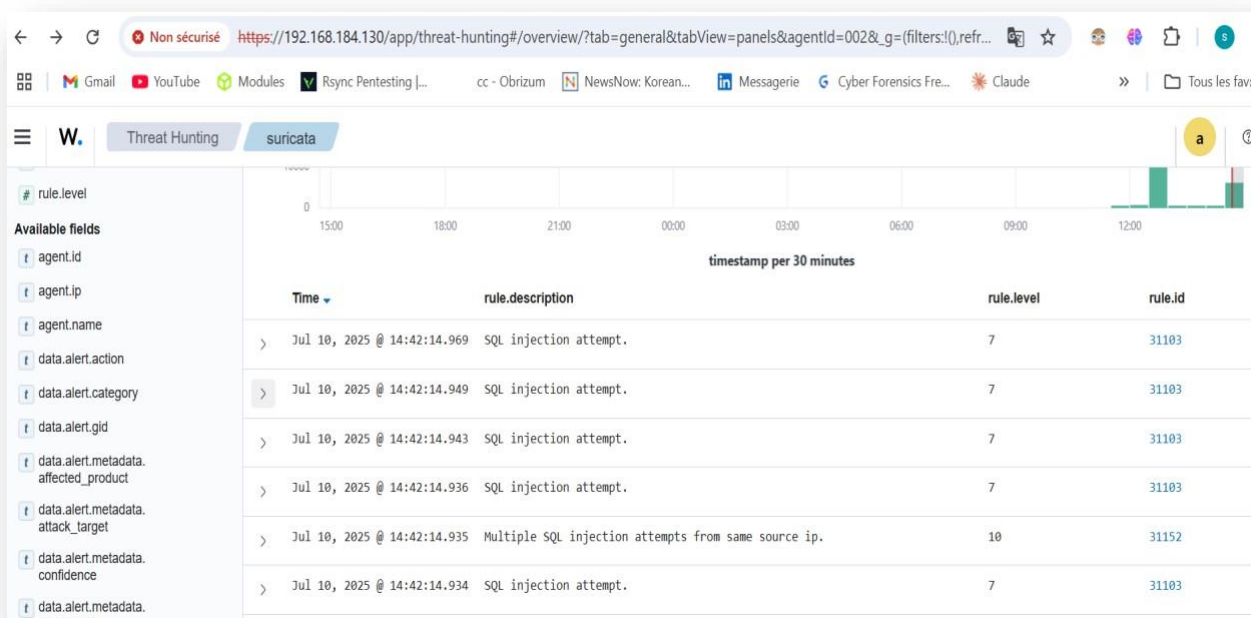
(kali@kali)~/var/www/html
$ sudo git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5373, done.
remote: Total 5373 (delta 0), reused 0 (delta 0), pack-reused 5373 (from 1)
Receiving objects: 100% (5373/5373), 2.57 MiB | 5.56 MiB/s, done.
Resolving deltas: 100% (2673/2673), done.
$ ls
DVWA index.html index.nginx-debian.html
$ ls -al
total 28
drwxr-xr-x 3 root root 4096 Jul 10 13:26 .
drwxr-xr-x 3 root root 4096 Jul 5 20:41 ..
drwxr-xr-x 12 www-data www-data 4096 Jul 10 13:26 DVWA
-rw-r--r-- 1 root root 10783 Jul 5 20:51 index.html
-rw-r--r-- 1 root root 615 Jul 5 20:47 index.nginx-debian.html
$ cd DVWA
$ sudo cp config/config.inc.php.dist config/config.inc.php
$ sudo nano config/config.inc.php
(kali@kali)~/var/www/html/DVWA
$ sudo mysql -u root
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.6.2-MariaDB-1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server
Copyright (c) 2000, 2019, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [(none)]> CREATE DATABASE dvwa;
Query OK, 1 row affected (0.039 sec)
MariaDB [(none)]> CREATE USER 'dvwauser'@'localhost' IDENTIFIED BY 'dvwapass';
Query OK, 0 rows affected (0.138 sec)
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwauser'@'localhost';
Query OK, 0 rows affected (0.020 sec)
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.036 sec)
MariaDB [(none)]> EXIT;
Bye

```

➤ Figure 18: Clonage du projet DVWA

3. Attaques simulées

Plusieurs types d'attaques ont été lancées manuellement (ou via des outils comme sqlmap, nikto, wpscan) pour tester la réactivité de l'IDS :



➤ Figure 19: Attaques simulées sur honeypots

Type d'attaque	Description	Détection
Injection SQL	Exploitation du paramètre de login	Alertes Suricata + VirusTotal
XSS (Cross-site Scripting)	Insertion de scripts dans les champs formulaire	Détection par règles personnalisées
Remote File Inclusion	Chargement de fichiers PHP distants	Détecté par Suricata
Bruteforce SSH / login	Utilisation de hydra sur le port 22	Détection par règle Wazuh 5710
Scan de vulnérabilités	Scan avec nmap -sV --script vuln	Alertes Suricata générées
Web Shell Upload	Téléchargement de backdoors PHP	FIM + VirusTotal + suppression auto

Toutes ces attaques ont été captées via Suricata, enrichies via Wazuh, et dans certains cas, vérifiées via VirusTotal et traitées automatiquement via `remove_threat.sh`.

4. Le mode IDS/IPS avec Suricata et Wazuh

Suricata, tout comme Snort, est un moteur de détection et de prévention d'intrusion (IDS/IPS) open-source. Dans notre projet, il joue un rôle central en analysant le trafic réseau en temps réel, en identifiant les comportements suspects à l'aide de règles personnalisées, et en générant des alertes vers le SIEM Wazuh pour une corrélation approfondie. En mode IPS, Suricata peut également bloquer activement les paquets malveillants selon les politiques définies.

Exemples de règles Suricata

- **Règle IDS (détection uniquement)**

```
alert icmp any any -> any any (msg:"Ping ICMP détecté"; sid:1000001;)
```

Cette règle génère une alerte lorsqu'une requête ICMP (ping) est détectée dans le réseau.

- **Règle IPS (détection + blocage)**

```
drop icmp any any -> any any (msg:"Ping ICMP bloqué"; sid:1000002;)
```

Celle-ci va non seulement générer une alerte, mais également bloquer le paquet ICMP au niveau du pare-feu.

Ces règles sont intégrées dans les fichiers de signatures de Suricata (/etc/suricata/rules/) et chargées lors du démarrage du service via :

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0
```

5. Vérification du bon fonctionnement des règles via des scénarios de test

Pour s'assurer que les règles Suricata fonctionnent correctement, nous avons effectué plusieurs tests en générant du trafic ciblé à partir d'une machine d'attaque vers notre honeypot DVWA :

Test 1 - Ping ICMP

- **Commande utilisée :** ping [IP_DVWA]
- **Résultat attendu :** Une alerte Suricata est générée (et blocage si règle IPS activée).
- **Objectif :** Détecter les tentatives de reconnaissance réseau.

Test 2 - Scan Nmap

- **Commande utilisée :** nmap -sS [IP_DVWA]
- **Résultat attendu :** Détection des paquets SYN suspects via une règle comme :

```
alert tcp any any -> any any (flags:S; msg:"Scan Nmap détecté"; sid:1000003;)
```

Test 3 - Connexion FTP

- **Commande utilisée** : ftp [IP_DVWA]
- **Règle Suricata** :

```
alert tcp any any -> any 21 (msg:"Connexion FTP détectée"; sid:1000004;)
```

- **Objectif** : S'assurer que les connexions non chiffrées sont surveillées.

Test 4 - Connexion SSH

- **Commande utilisée** : ssh user@[IP_DVWA]
- **Règle Suricata** :

```
alert tcp any any -> any 22 (msg:"Connexion SSH détectée"; sid:1000005;)
```

- **Objectif** : Détecter les tentatives d'accès SSH, notamment en cas de brute force.

Chaque test a permis de valider la réactivité du moteur de détection et son intégration avec Wazuh. Les alertes générées ont été automatiquement remontées dans le Wazuh Dashboard, enrichies par la corrélation des logs, et dans certains cas, analysées via l'API VirusTotal.

6. Mode IPS et complémentarité avec iptables

En mode IPS, Suricata peut être configuré pour fonctionner en ligne ("inline") et agir comme un pare-feu applicatif. Il peut ainsi rejeter des paquets en fonction des signatures détectées, ce qui renforce la posture défensive du système.

De plus, dans notre projet, le pare-feu iptables a été utilisé de façon complémentaire :

- Pour bloquer dynamiquement des IP suspectes détectées par Wazuh.
- Pour journaliser certaines requêtes, notamment les connexions HTTP suspectes.
- Pour isoler rapidement la machine honeypot ou l'agent lors d'attaques avérées.

En résumé, l'utilisation de Suricata en mode IDS/IPS, couplé avec Wazuh pour la gestion centralisée des alertes, nous a permis de détecter, analyser et bloquer efficacement .

IV. Intégration de Wazuh Agent et Manager :

Après avoir installé et configuré Suricata pour détecter les attaques réseau sur notre première machine (Kali Linux), nous avons intégré la solution Wazuh pour centraliser, analyser et corréler les alertes de sécurité générées à la fois par Suricata et par d'autres sources du système (logs, fichiers sensibles, etc.). Cette étape est essentielle pour avoir une vue centralisée et enrichie des activités suspectes sur l'ensemble de notre environnement.

1. Objectif de Wazuh dans notre projet IDS

Wazuh est une plateforme open source de détection d'intrusion HIDS (Host-based Intrusion Detection System), de surveillance de l'intégrité des fichiers, d'analyse de logs, et de réponse automatisée aux incidents. Dans notre architecture :

- **Le Wazuh Manager** collecte, analyse et corrèle les données.
- **Le Wazuh Agent**, installé sur les machines surveillées (comme notre VM Kali), remonte les événements locaux.
- Le **Dashboard Web** (basé sur Kibana) permet une visualisation claire des alertes et événements.

2. Installation du Wazuh Manager sur une nouvelle VM

a. Préparation de la machine

Nous avons utilisé une nouvelle machine virtuelle dédiée (Ubuntu Server recommandé) afin d'éviter les conflits (car nous avons appris qu'il ne faut pas installer le manager et l'agent sur la même machine de test).

b. Installation via script automatique

Nous avons téléchargé et exécuté le script officiel :

```
[kali@kali]~$ sudo ./wazuh-install.sh -e --ignore-check
06/07/2025 17:19:47 INFO: Starting Wazuh installation assistant. Wazuh version: 4.8.2
06/07/2025 17:19:47 INFO: Verbose logging redirected to /var/log/wazuh-install.log
06/07/2025 17:19:50 WARNING: Hardware and system checks ignored.
06/07/2025 17:19:58 INFO: Wazuh web interface port will be 443.
06/07/2025 17:20:05 INFO: --- Dependencies ---
06/07/2025 17:20:02 INFO: Installing apt-transport-https.
06/07/2025 17:20:06 INFO: Installing software-properties-common.
06/07/2025 17:20:26 INFO: Wazuh repository added.
06/07/2025 17:20:26 INFO: --- Configuration files ---
06/07/2025 17:20:26 INFO: Generating configuration files.
06/07/2025 17:20:26 INFO: Generating the root certificate.
06/07/2025 17:20:26 INFO: Generating Admin certificates.
06/07/2025 17:20:26 INFO: Generating Wazuh indexer certificates.
06/07/2025 17:20:26 INFO: Generating Filebeat certificates.
06/07/2025 17:20:27 INFO: Generating Wazuh dashboard certificates.
06/07/2025 17:20:27 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
06/07/2025 17:20:27 INFO: Starting Wazuh indexer installation.
06/07/2025 17:23:02 INFO: Wazuh indexer installation finished.
06/07/2025 17:23:02 INFO: Wazuh indexer post-install configuration finished.
06/07/2025 17:23:02 INFO: Starting service wazuh-indexer.
06/07/2025 17:23:49 INFO: wazuh-indexer service started.
06/07/2025 17:23:49 INFO: Initializing Wazuh indexer cluster security settings.
```

➤ Figure 20: installation de Wazuh server

Ce script a automatiquement installé :

- Le Wazuh Manager
- Le Dashboard web (Kibana/Elasticsearch)
- Les configurations nécessaires de base.

➤ Démarrage de Wazuh Manager :

Le Wazuh Manager a été lancé pour gérer les alertes et analyser les logs provenant de l'agent installé sur la machine honeypot , pour s'assurer que le Wazuh Manager est correctement opérationnel, nous avons vérifié son statut avec la commande suivante :

```
[kali@kali]~$ sudo systemctl status wazuh-dashboard
● wazuh-dashboard.service - wazuh-dashboard
   Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-07-07 11:57:56 +01:37min ago
     Invocation: 43a8d7a3908443e1beca0a418ee0c671
       Main PID: 68443 (node)
         Tasks: 11 (limit: 3876)
       Memory: 187.9M (peak: 283.6M)
          CPU: 36.411s
      CGroup: /system.slice/wazuh-dashboard.service
              └─68443 /usr/share/wazuh-dashboard/node/bin/node --max-http-header-size=65536 --unhandled-rejection=warn /usr/share/wazuh-dashboard/src/cli/dist

Jul 07 12:11:17 kali opensearch-dashboards[68443]: {"type":"response","timestamp":"2025-07-07T11:11:17Z","tags":[],"pid":68443,"method":"get","statusCode":200,"req":{"url":"/ui/logos/opensearch_mark_on_light
Jul 07 12:11:17 kali opensearch-dashboards[68443]: {"type":"response","timestamp":"2025-07-07T11:11:17Z","tags":[],"pid":68443,"method":"post","statusCode":200,"req":{"url":"/api/request","method":"post","he
Jul 07 12:11:17 kali opensearch-dashboards[68443]: {"type":"response","timestamp":"2025-07-07T11:11:17Z","tags":[],"pid":68443,"method":"post","statusCode":200,"req":{"url":"/api/request","method":"post","he
Jul 07 12:11:17 kali opensearch-dashboards[68443]: {"type":"response","timestamp":"2025-07-07T11:11:17Z","tags":[],"pid":68443,"method":"post","statusCode":200,"req":{"url":"/api/request","method":"post","he
Jul 07 12:11:17 kali opensearch-dashboards[68443]: {"type":"response","timestamp":"2025-07-07T11:11:17Z","tags":[],"pid":68443,"method":"post","statusCode":200,"req":{"url":"/api/request","method":"post","he
Jul 07 12:15:01 kali opensearch-dashboards[68443]: {"type":"log","timestamp":"2025-07-07T11:15:01Z","tags":["error","opensearch","data"],"pid":68443,"message":["index.create_block_exception]: blocked by: {\"r
Jul 07 12:15:01 kali opensearch-dashboards[68443]: {"type":"log","timestamp":"2025-07-07T11:15:01Z","tags":["error","opensearch","data"],"pid":68443,"message":["index.create_block_exception]: blocked by: {\"r
Jul 07 12:15:01 kali opensearch-dashboards[68443]: {"type":"log","timestamp":"2025-07-07T11:15:01Z","tags":["error","plugins","wazuh","cron-scheduler"],"pid":68443,"message":["\\error\\":{\\root_cause\\":{\\t

[kali@kali]~$ sudo systemctl start wazuh-manager
[kali@kali]~$ sudo systemctl status wazuh-manager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-07-07 11:57:58 +01:38min ago
     Invocation: 348e83d16f064ef190597f249f0f5a3
       Tasks: 153 (limit: 3876)
       Memory: 1G (peak: 1.1G, swap: 3.5M, swap peak: 5.5M)
          CPU: 2min 36.775s
      CGroup: /system.slice/wazuh-manager.service
              └─67122 /usr/ossec/framework/python/bin/python3 /usr/ossec/api/scripts/wazuh-api.py
                 67623 /usr/ossec/framework/python/bin/python3 /usr/ossec/api/scripts/wazuh-api.py
                 67626 /usr/ossec/framework/python/bin/python3 /usr/ossec/api/scripts/wazuh-api.py
                 67629 /usr/ossec/framework/python/bin/python3 /usr/ossec/api/scripts/wazuh-api.py
                 67679 /usr/ossec/bin/wazuh-auth
                 67782 /usr/ossec/bin/wazuh-mn
                 67713 /usr/ossec/bin/wazuh-syscheckd
                 67747 /usr/ossec/bin/wazuh-analysisd
                 67766 /usr/ossec/bin/wazuh-syslogd
                 67770 /usr/ossec/bin/wazuh-remoted
                 67862 /usr/ossec/bin/wazuh-logcollector
                 67884 /usr/ossec/bin/wazuh-maild
                 67922 /usr/ossec/bin/wazuh-moduled
```

- Figure 21 : Démarrage de Wazuh server

Le Wazuh Manager est opérationnel et fonctionne correctement. Tous les modules essentiels sont lancés, ce qui garanti

c. Accès au dashboard

Une fois installé, nous avons accédé au dashboard via l'URL :

- `https://<ip_vm_manager>`

Nous avons vérifié l'accès et créé un utilisateur administrateur.

2. Installation et configuration du Wazuh Agent :

a. Préparation et nettoyage

Dans un premier temps, nous avons installé le manager sur Kali par erreur, mais cela a été corrigé. Après avoir supprimé le manager, nous avons procédé à l'installation propre de l'agent seul.

b. Installation:

Sur la machine où le honeypot est déployé (qui contient également les outils comme Suricata et syslog-ng), nous avons installé Wazuh Agent pour collecter les logs générés par ces outils. L'installation s'effectue via les commandes suivantes :

```
All packages are up to date.
(kali@kali) [~/siem]
$ sudo apt install wazuh-agent

Installing:
wazuh-agent

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 12.0 MB
  Space needed: 44.0 MB / 40.9 GB available

Get:1 https://packages.wazuh.com/4.x/apt/stable/main amd64 wazuh-agent amd64 4.12.0-1 [12.0 MB]
Fetched 12.0 MB in 8s (1,470 kB/s)
Preconfiguring packages ...
Selecting previously unselected package wazuh-agent.
(Reading database ... 422269 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.12.0-1_amd64.deb ...
Unpacking wazuh-agent (4.12.0-1) ...
Setting up wazuh-agent (4.12.0-1) ...
```

➤ Figure 22 :Installation du Wazuh Agent

c. Configuration de l'agent

Après l'installation, il est crucial de configurer l'agent pour qu'il envoie les logs au Wazuh Manager. Cela se fait en modifiant le fichier de configuration `/var/ossec/etc/ossec.conf`, où l'adresse du manager est définie comme suit :

```
<ossec_config>
  <client>
    <server>
      <address>192.168.184.130</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>kali, kali2025, kali2025.2</config-profile>
    <notify_time>10</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
  </client>
```

➤ Figure 23: Configuration de l'agent

d. Enregistrement sur le manager

Nous avons ensuite exécuté sur le manager :

➤ `/var/ossec/bin/manage_agents`

Pour ajouter l'agent et générer une clé d'authentification, que nous avons ensuite importée dans l'agent via :

➤ `/var/ossec/bin/agent-auth -m IP_DU_MANAGER`

e. Vérification du Bon Fonctionnement de l'Agent :

Afin de vérifier que l'agent fonctionne correctement et que les logs sont bien collectés, nous avons utilisé la commande suivante pour vérifier son état :



```
(kali) kali ~  
$ sudo systemctl status wazuh-agent  
  
● wazuh-agent.service - Wazuh agent  
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)  
   Active: active (running) since Tue 2025-04-08 21:46:57 +01; 2 days ago  
     Invocation: d41e93f92c994fd282f9b9de604bf130  
   Process: 320844 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)  
    Tasks: 34 (limit: 4537)  
  Memory: 366.6M (peak: 1.1G, swap: 4.5M, swap peak: 4.6M)  
    CPU: 21min 42.541s  
   CGroup: /system.slice/wazuh-agent.service  
           └─320866 /var/ossec/bin/wazuh-execd  
             └─320886 /var/ossec/bin/wazuh-agentd  
               └─320916 /var/ossec/bin/wazuh-syscheckd  
                 └─320926 /var/ossec/bin/wazuh-logcollector  
                   └─320933 /var/ossec/bin/wazuh-logcollector
```

➤ Figure 24: Démarrage de Wazuh agent

b. Ajout de l'agent sur le Manager

Sur le Wazuh Manager, nous avons utilisé la commande :

- `/var/ossec/bin/manage_agents`

Puis suivi les étapes :

1. Taper A pour ajouter un nouvel agent.
2. Donner un nom à l'agent (ex. KALI-SURICATA) et valider.
3. Copier la clé d'enregistrement générée.

c. Enregistrement de l'agent depuis la machine Kali

Sur la machine Kali (où l'agent est installé), nous avons exécuté :

```
sudo /var/ossec/bin/agent-auth -m <IP_DU_MANAGER>
```

Cela permet à l'agent de s'enregistrer automatiquement avec le manager.

4. Intégration avec Suricata :

Suricata est configuré par défaut pour écrire ses alertes dans un fichier JSON :

- /var/log/suricata/eve.json

Nous avons simplement configuré l'agent Wazuh pour surveiller ce fichier en ajoutant dans /var/ossec/etc/ossec.conf :

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

5. Visualisation des alertes dans le dashboard

Une fois l'agent opérationnel et Suricata fonctionnel, les alertes générées (scan, injection, brute force, etc.) sont automatiquement envoyées au Wazuh Manager, puis visibles dans le dashboard.

Nous avons utilisé les filtres du dashboard Wazuh pour :

- Suivre les événements de sécurité.
- Vérifier les logs Suricata.
- Visualiser les tentatives d'attaques sur la machine honeypot DVWA.

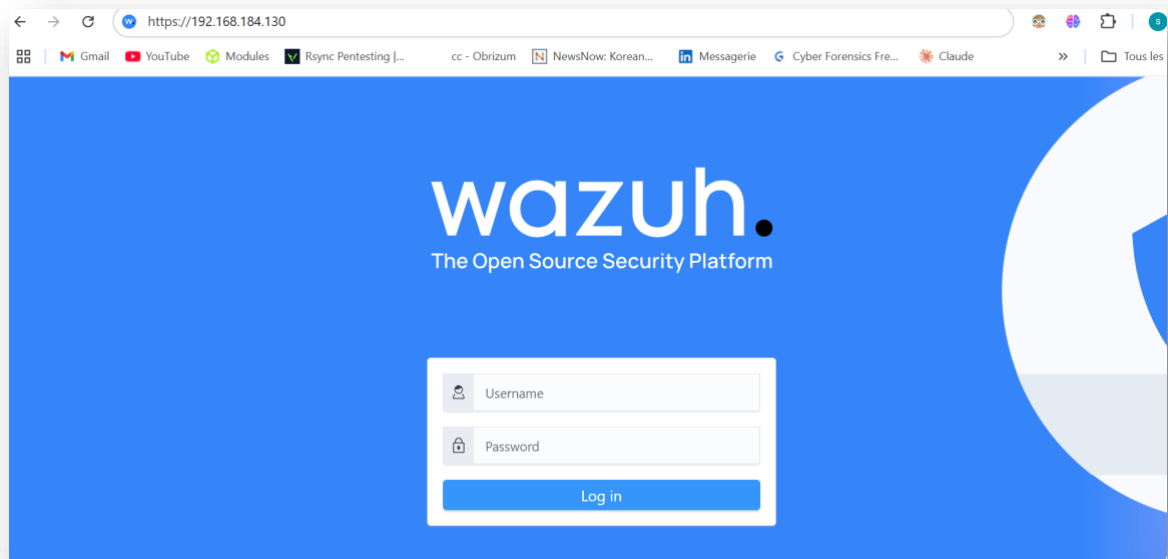
6. Résultat des tests

Pour tester notre intégration, nous avons :

- Lancé un scan Nmap sur la machine Kali.
- Simulé des injections sur DVWA.
- Observé la génération d'alertes dans /var/log/suricata/eve.json.
- Constaté leur apparition en temps réel dans Wazuh Dashboard.

V. Accès à l'Interface Graphique de Wazuh Manager :

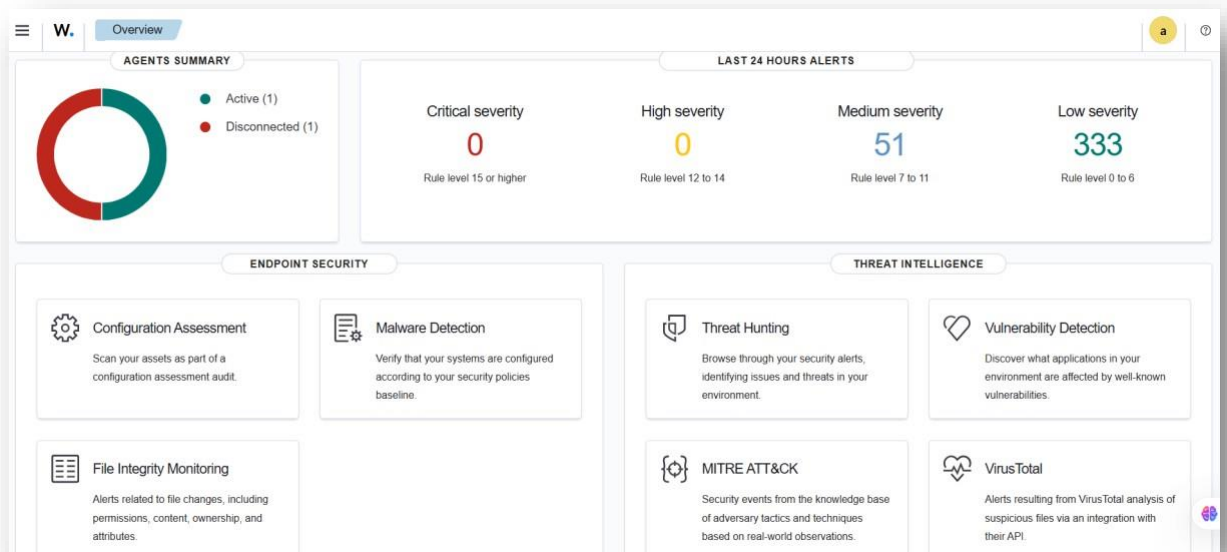
L'interface graphique de Wazuh Manager est accessible via un navigateur web à l'adresse suivante :



➤ Figure 25: L'interface graphique de Wazuh server

1. Présentation du Dashboard Wazuh

Après authentification sur l'interface web de Wazuh, nous accédons à un dashboard centralisé offrant une vue d'ensemble de l'état de sécurité de notre environnement. Ce tableau de bord est divisé en plusieurs sections clés :



➤ Figure 26 :Présentation du Dashboard Wazuh

a. Agents Summary (Résumé des Agents)

- **Rôle :** Affiche la liste des agents connectés au manager.
- **Importance :** Permet de vérifier rapidement si les agents sont correctement enregistrés et actifs.
- **Observation :** Dans notre cas, aucun agent n'est encore ajouté (message : *"No agents were added to this manager"*).

b. Last 24 Hours Alerts (Alertes des 24 dernières heures)

- **Rôle :** Présente un résumé des alertes générées, classées par niveau de gravité :
 - **Critical severity** (niveau 15+)
 - **High severity** (niveau 12 à 14)
 - **Medium severity** (niveau 7 à 11)
 - **Low severity** (niveau 0 à 6)

- **Importance** : Utile pour détecter rapidement les incidents de sécurité critiques et suivre l'évolution des menaces.

c. Endpoint Security (Sécurité des hôtes surveillés)

Cette section regroupe les modules essentiels liés à la protection des machines :

- **Configuration Assessment** : Audit de la configuration des systèmes selon les politiques de sécurité.
- **Malware Detection** : Vérifie la présence d'éléments malveillants ou de mauvaises configurations.
- **File Integrity Monitoring (FIM)** : Surveille les modifications de fichiers sensibles (droits, contenu, propriétaire...).

d. Threat Intelligence (Renseignement sur les menaces)

- **Threat Hunting** : Permet une analyse proactive des événements pour identifier les comportements suspects.
- **Vulnerability Detection** : Détecte les vulnérabilités connues dans l'environnement.
- **MITRE ATTsCK** : Corrèle les événements de sécurité avec les tactiques et techniques adverses du framework MITRE.
- **VirusTotal** : Intègre les résultats d'analyse de fichiers suspects à l'aide de l'API VirusTotal.

e. Security Operations (Conformité et réglementation)

- **PCI DSS** : Pour les entreprises traitant des données de cartes bancaires (conformité PCI).
- **GDPR** : Pour la conformité avec la réglementation européenne sur la protection des données personnelles.

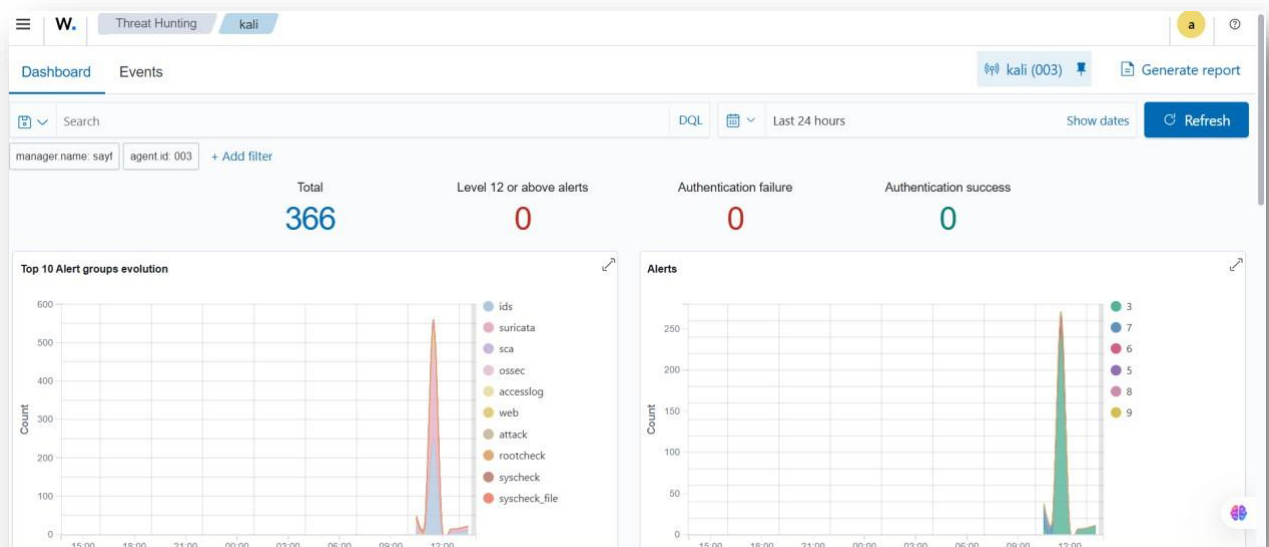
f. Cloud Security (Sécurité Cloud)

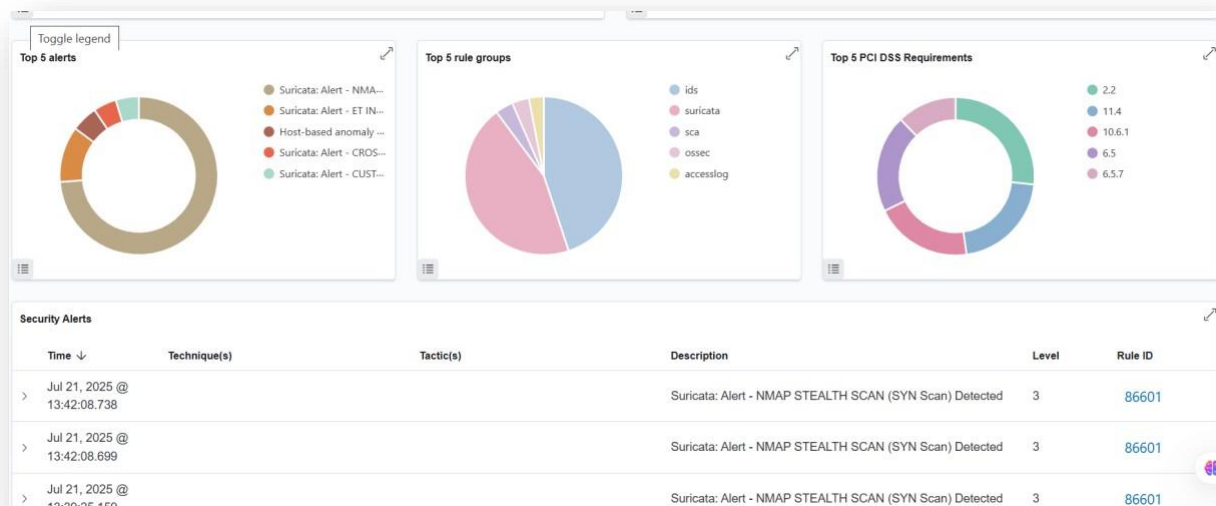
- **Docker** : Surveillance des conteneurs Docker (création, démarrage, arrêt, etc.).
- **Amazon Web Services (AWS)** : Intégration et suivi des services AWS pour détecter les activités suspectes.

2. AGENT KALI :

Après avoir présenté les différentes fonctionnalités offertes par le tableau de bord principal de Wazuh, nous allons maintenant analyser plus en détail les données de sécurité remontées par un agent, à travers l'interface de supervision en temps réel.

Analyse de la vue "Dashboard" de Wazuh (agent : Kali) :





➤ Figure 27: interfac des événements de l'agent

Cette interface présente un aperçu des événements de sécurité détectés pour l'agent nommé "kali" :

- **Total des alertes :** 366 événements de sécurité enregistrés.
- **Alertes critiques (niveau 12 ou plus) :** 0 – aucun événement critique n'a été détecté.
- **Échecs d'authentification :** 0 – aucun échec recensé.
- **Réussites d'authentification :** 0 – indiquant des accès utilisateur valides.

Évolution du niveau d'alerte :

Un graphique montre la variation des alertes dans le temps, avec des pics notables à certaines heures. Les couleurs représentent différents niveaux de gravité (ex. : niveau 3, 7, 6, 9)

A. Top 5 Alerts

Cette section montre les alertes de sécurité les plus fréquentes :

- Suricata Alert - NMA... : Probablement des alertes liées à des scans réseau détectés par Suricata (comme le scan NMAP visible dans le détail)
- Suricata Alert - ET IN... : Alertes provenant des règles Emerging Threats (ET)
- Host-based anomaly : Anomalie détectée au niveau d'un hôte
- Suricata Alert - CROS... et **CUST...** : Autres alertes Suricata (peut-être des attaques cross-site ou des règles personnalisées)

B. 5 Rule Groups

Indique les groupes de règles les plus déclenchés :

- Suricata** : Alertes du moteur IDS/IPS Suricata
- Ids: Détection d'intrusions
- Sca : Security Configuration Assessment (vérification de conformité)
- Ossec : Alertes du moteur HIDS original
- Accesslog: Analyses des logs d'accès

C. Top 5 PCI DSS Requirements

Montre les exigences PCI DSS les plus concernées par les alertes :

- 2.2 : Configuration sécurisée des systèmes
- 11.4 : Surveillance des accès réseau
- 10.6.1 : Revue des logs
- 6.5 : Gestion des vulnérabilités
- 6.3.7 : Sécurité des applications

D. Security Alerts (Détail)

Tableau détaillant les alertes récentes :

- Plusieurs occurrences d'une alerte "NMAP STEALTH SCAN (SYN Scan) Detected"
- Niveau 3 (gravité moyenne)
- Rule ID 86601 (règle Suricata pour détecter les scans furtifs)
- Ces alertes indiquent qu'un scan réseau de type SYN (première étape d'un scan NMAP) a été détecté sur le réseau

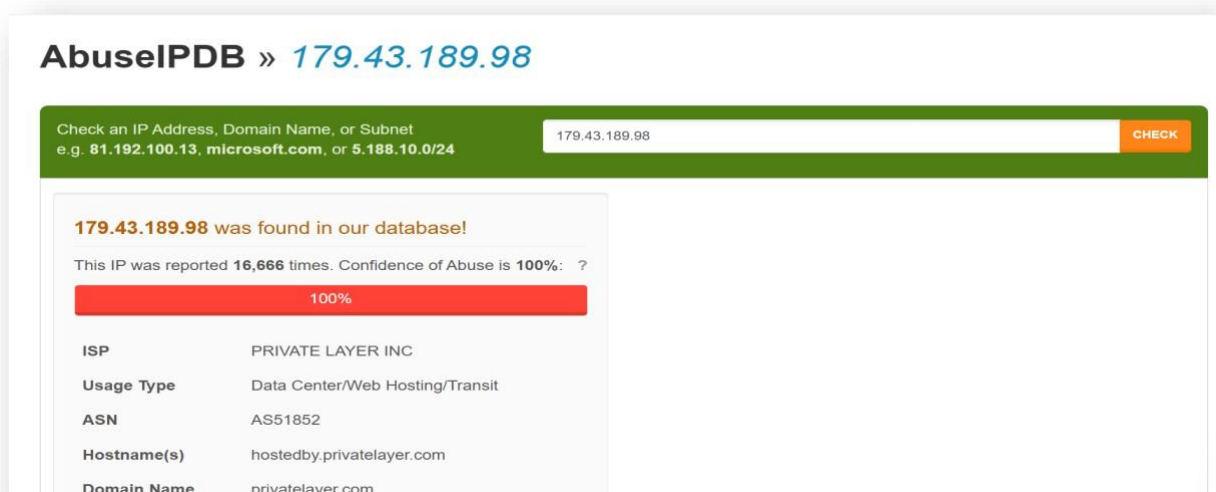
E. Interprétation globale

Le tableau de bord montre une activité notable de scanning réseau (potentielle reconnaissance par un attaquant), avec plusieurs détections de scans NMAP furtifs. Les alertes sont principalement générées par Suricata (IDS/IPS) et concernent particulièrement les exigences PCI DSS liées à la surveillance réseau et la configuration sécurisée.

La répétition des mêmes alertes de scan suggère qu'une investigation plus poussée serait nécessaire pour identifier la source et déterminer s'il s'agit d'un test légitime ou d'une activité malveillante.

VI. Intégration de l'API AbuseIPDB à notre IDS

Dans le cadre de notre projet IDS, nous avons intégré l'API publique d'**AbuseIPDB** afin de renforcer les capacités de détection et de réaction face aux adresses IP malveillantes. L'objectif était d'ajouter une couche d'intelligence externe à notre système, en vérifiant automatiquement la réputation des IP sources détectées dans les événements d'alerte générés par Suricata et transmis à Wazuh.



➤ Figure 28: AbuseIPDB site web

Pour cela, nous avons développé un script Python personnalisé situé dans le répertoire des **active-responses** de Wazuh (/var/ossec/active-response/bin/), nommé `abuseipdb_checker.py`.

```

GNU nano 8.4 /var/ossec/active-respons
#!/usr/bin/env python3
import sys
import json
import requests
import subprocess

API_KEY = "TA_CLE_ABUSEIPDB"
THRESHOLD_SCORE = 50

def check_ip(ip):
    url = f"https://api.abuseipdb.com/api/v2/check"
    headers = {
        "Accept": "application/json",
        "Key": API_KEY
    }
    params = {
        "ipAddress": ip,
        "maxAgeInDays": 90
    }
    try:
        response = requests.get(url, headers=headers, params=params)
        data = response.json()
        score = data["data"]["abuseConfidenceScore"]
        return score
    except Exception as e:
        print(f"Erreur API: {e}")
        return 0

def block_ip(ip):
    subprocess.run(["iptables", "-A", "INPUT", "-s", ip, "-j", "DROP"])
    print(f"[+] IP {ip} bloquée avec iptables")

if __name__ == "__main__":
    alert_data = json.load(sys.stdin)
    ip = alert_data.get("srcip", "")
    if ip:
        score = check_ip(ip)
        print(f"Score AbuseIPDB: {score}")
        if score >= THRESHOLD_SCORE:
            block_ip(ip)

```

➤ Figure 29: script d' AbuseIPDB

Ce script récupère les IP à partir des logs d'alerte, interroge l'API d'AbuseIPDB avec une clé d'API personnelle, puis décide, en fonction du score de réputation retourné, d'appliquer ou non une contre-mesure.

Lorsque le score dépasse un certain seuil (par exemple 90), le script déclenche une réponse active : un bannissement temporaire de l'adresse IP par iptables, pendant une durée définie (30 secondes dans notre cas), avant de la débloquent automatiquement. Cela permet de bloquer dynamiquement des IP connues pour des comportements abusifs (scan, brute-force, exploitation, etc.) tout en limitant les faux positifs. Cette approche améliore la réactivité de l'IDS et rend notre système capable d'agir automatiquement face aux menaces connues au niveau mondial.

```

(root@kali)-[/home/kali/siem]
# cat test.json | sudo /var/ossec/active-response/bin/abuseipdb_checker.py
+] Score AbuseIPDB: 0

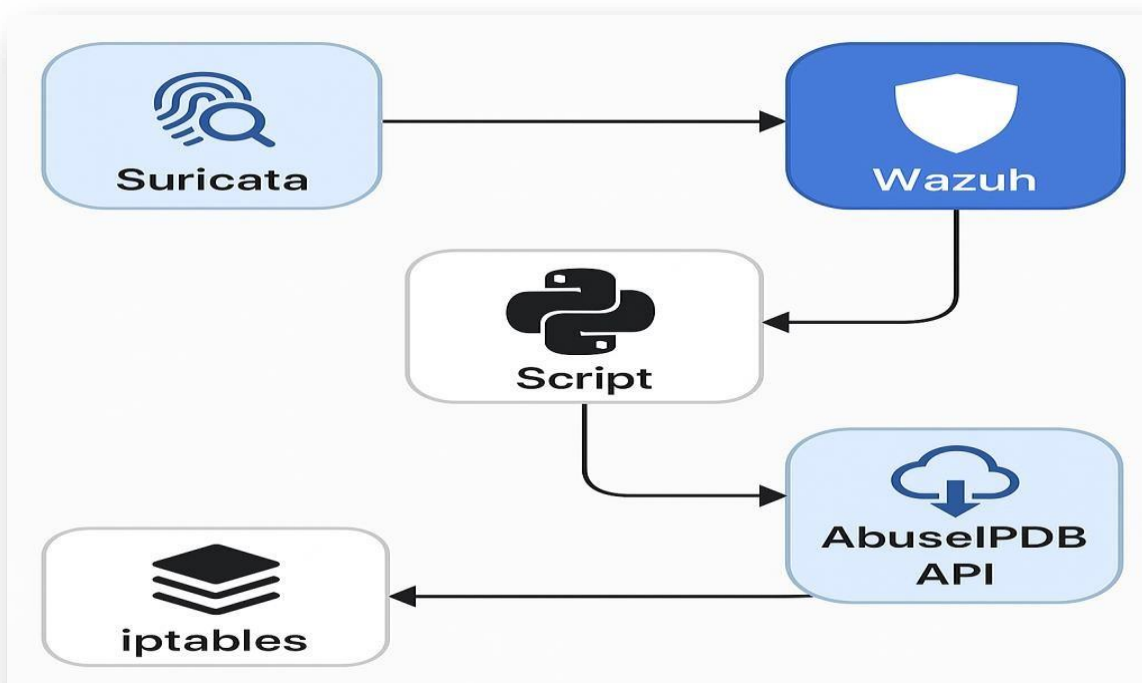
(root@kali)-[/home/kali/siem]
# nano test.json

(root@kali)-[/home/kali/siem]
# cat test.json | sudo /var/ossec/active-response/bin/abuseipdb_checker.py
+] Score AbuseIPDB: 100
+] Blocage temporaire de l'IP 179.43.189.98
+] IP 179.43.189.98 débloquée après 30 secondes

```

➤ Figure 30: test de script d' AbuseIPDB

L'appel au script est déclenché depuis les règles personnalisées (local_rules.xml) sur le manager Wazuh, lorsque certaines alertes critiques sont générées. L'ensemble du processus a été testé avec succès en simulant des attaques depuis une IP malveillante référencée dans la base AbuseIPDB, ce qui a permis de valider l'efficacité de l'intégration.



➤ Figure 31: schema du travail de api AbuseIPDB

Blocage automatique avec iptables :

Une fois une IP considérée comme malveillante, le script exécute une commande iptables pour bloquer le trafic entrant en provenance de cette adresse.

```
def block_temporarily(ip):
    print(f"[+] Blocage temporaire de l'IP {ip}")
    subprocess.run(["iptables", "-I", "INPUT", "-s", ip, "-j", "DROP"])
    time.sleep(BLOCK_DURATION)
    subprocess.run(["iptables", "-D", "INPUT", "-s", ip, "-j", "DROP"])
    print(f"[+] IP {ip} débloquée après {BLOCK_DURATION} secondes")

if __name__ == "__main__":
```

VII. Intégration de l'API VirusTotal dans Wazuh et Suricata

1. Pourquoi intégrer VirusTotal ?

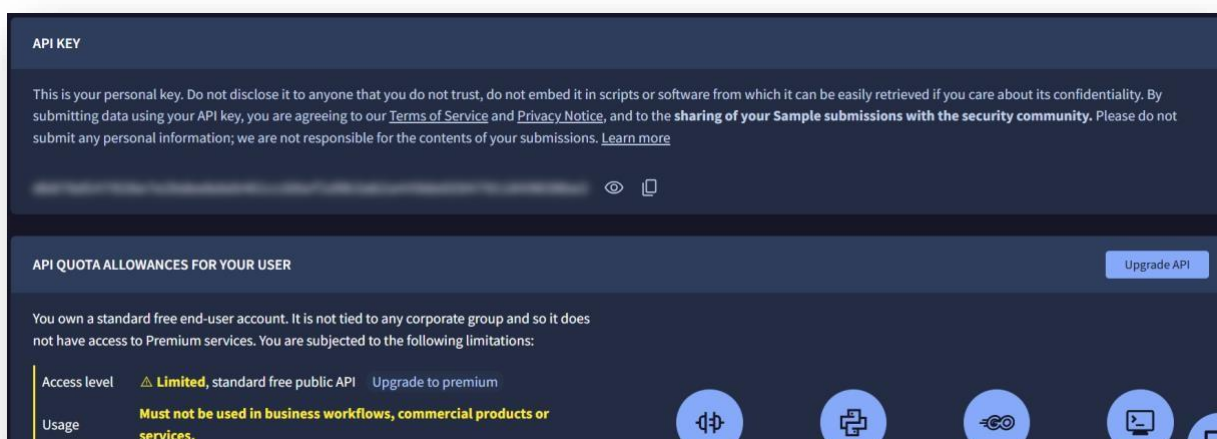
Intégrer VirusTotal à notre système de détection (Wazuh) permet :

- De valider automatiquement les fichiers suspects détectés par Suricata.
- D'obtenir une réputation des fichiers, hash ou URLs.
- D'enrichir les alertes générées par les règles de détection avec des informations contextuelles.

2. Étapes de l'intégration

1. Obtention de la clé API VirusTotal :

- Création d'un compte sur [virustotal.com](https://www.virustotal.com) pour générer une clé API gratuite.



➤ Figure 33 :VirusTotal site web

Activation du module VirusTotal dans Wazuh :

- On a modifié le fichier de configuration Wazuh (/var/ossec/etc/ossec.conf) pour activer l'intégration :

```
<!-- local file -->
<integration>
  <name>virustotal</name>
  <api_key>*****547928e7e2bdeebdeb461ccb9af1d9b3ab2a449de65947911049030be2</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>

</ossec_config>
```

➤ Figure 34: integration de VirusTotal dans Wazuh

2. Création de règles de détection ciblant les événements à envoyer à VirusTotal :

- Par exemple, toute alerte contenant un hash SHA256 d'un fichier inconnu peut être envoyée à VirusTotal.
- Exemple de règle dans /var/ossec/etc/rules/local_rules.xml:

```
<group name="virustotal,">
  <rule id="100200" level="10">
    <decoded_as>json</decoded_as>
    <description>VirusTotal: Malware detected</description>
    <options>alert_by_email</options>
    <match>virustotal</match>
  </rule>
</group>
```

➤ Figure 34: integration de VirusTotal dans Wazuh

3. Lien avec Suricata :

- Suricata détecte certains fichiers via les signatures (ex : téléchargements ou uploads suspects).
- Les logs générés sont transmis à Wazuh via l'agent.

- Si un hash est extrait, il est automatiquement envoyé à VirusTotal pour enrichissement.

5. Surveillance des fichiers critiques (File Integrity Monitoring - FIM)

5.1 Objectif

Le module **Syscheck** de Wazuh permet de surveiller l'intégrité des fichiers sensibles du système. Il détecte les modifications non autorisées (création, suppression, modification) sur des fichiers ou dossiers spécifiés.

5.2 Configuration dans Wazuh Agent

Dans le fichier de configuration de l'agent Wazuh (/var/ossec/etc/ossec.conf), nous avons activé la surveillance de plusieurs répertoires critiques :

```

#-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  #-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  #-- Directories to check (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
  <directories check_all="yes" realtime="yes">/home/kali/Downloads</directories>
  <directories check_all="yes" realtime="yes">/tmp</directories>
  <directories check_all="yes" realtime="yes">/home</directories>

  #-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>

  #-- File types to ignore -->
  <ignore type="sregex">.log$|.swp$</ignore>

  #-- Check the file, but never compute the diff -->
  <nodiff>/etc/ssl/private.key</nodiff>

```

➤ Figure 35: les répertoires à surveiller

Cette configuration permet :

- De détecter les changements de permissions.

- De repérer des ajouts de fichiers potentiellement malveillants (ex : scripts inconnus dans /bin , /etc, /home ...).
- D'obtenir des alertes en temps réel lors de la modification de fichiers système.

5.3 Protection contre les menaces

En cas de détection :

- Une alerte est générée dans l'interface du **Wazuh Dashboard**.
- Si un fichier est modifié, son hash peut être envoyé automatiquement à VirusTotal.
- On peut lier l'alerte à une action de mitigation automatique (par exemple, suppression, isolement de la machine, ou alerte par mail/Slack).

Time ▾	syscheck.path	syscheck.event	rule.description	rule.level	rule.id
Jul 16, 2025 @ 12:40:10.349	/tmp/malware9	deleted	File deleted.	7	553
Jul 16, 2025 @ 12:40:07.748	/tmp/malware9	added	File added to the system.	5	554
Jul 16, 2025 @ 12:39:48.822	/tmp/malware8	deleted	File deleted.	7	553
Jul 16, 2025 @ 12:39:44.195	/tmp/malware8	added	File added to the system.	5	554
Jul 16, 2025 @ 12:38:57.734	/tmp/malware7	deleted	File deleted.	7	553
Jul 16, 2025 @ 12:38:52.344	/tmp/malware7	added	File added to the system.	5	554

5.4 Script de suppression automatique des fichiers malveillants

Afin de renforcer la réponse automatique aux menaces détectées, nous avons intégré un script personnalisé nommé `remove_threat.sh`. Ce script a

pour rôle de supprimer automatiquement tout fichier identifié comme malveillant par Wazuh après une analyse via VirusTotal.

Voici les points clés de cette implémentation :

- Le script est placé dans un répertoire surveillé et autorisé à l'exécution automatique par Wazuh (active-response/bin).
- Lorsqu'un fichier est détecté comme malveillant (par exemple, suite à une analyse VirusTotal confirmant un taux élevé de détection), une alerte est générée et déclenche le script `remove_threat.sh`.
- Le script lit l'alerte, identifie le chemin du fichier suspect, puis supprime le fichier de manière sécurisée tout en générant un log pour archivage.

4. Create the `/var/ossec/active-response/bin/remove-threat.sh` active response script to remove malicious files from the endpoint:

```
#!/bin/bash

LOCAL=`dirname $0`;
cd $LOCAL
cd ../

PWD=`pwd`

read INPUT_JSON
FILENAME=$(echo $INPUT_JSON | jq -r .parameters.alert.data.virustotal.source.file)
COMMAND=$(echo $INPUT_JSON | jq -r .command)
LOG_FILE="${PWD}/../logs/active-responses.log"

#----- Analyze command -----#
if [ ${COMMAND} = "add" ]
then
    # Send control message to execd
    printf '{"version":1,"origin":{"name":"remove-threat","module":"active-response"},"command":"check_keys", "parameters":{"keys'

    read RESPONSE
    COMMAND2=$(echo $RESPONSE | jq -r .command)
    if [ ${COMMAND2} != "continue" ]
    then
        echo "`date '+%Y/%m/%d %H:%M:%S'` $0: $INPUT_JSON Remove threat active response aborted" >> ${LOG_FILE}
        exit 0;
    fi
fi

# Removing file
rm -f $FILENAME
if [ $? -eq 0 ]; then
    echo "`date '+%Y/%m/%d %H:%M:%S'` $0: $INPUT_JSON Successfully removed threat" >> ${LOG_FILE}
else
    echo "`date '+%Y/%m/%d %H:%M:%S'` $0: $INPUT_JSON Error removing threat" >> ${LOG_FILE}
fi

exit 0;
```

➤ Figure 36: Script de suppression des fichiers malveillants

Pour activer ce comportement, nous avons configuré la réponse active dans Wazuh (/var/ossec/etc/ossec.conf) :

```
<active-response>
  <command>remove-threat</command>
  <location>local</location>
  <rules_id>100500</rules_id> <!-- règle personnalisée VirusTotal -->
</active-response>
```

Cette automatisation permet d'agir rapidement et efficacement contre les menaces, réduisant considérablement le temps de réaction et les risques de propagation.

6. Exemple d'utilisation en conditions réelles

Lors de nos tests avec l'application vulnérable **DVWA** sur un honeypot :

- Suricata a détecté une tentative d'injection de fichier malveillant.
- Le fichier a été enregistré dans /tmp, surveillé par le module FIM de Wazuh.
- Wazuh a généré une alerte, a extrait le hash du fichier et l'a envoyé à VirusTotal.
- VirusTotal a confirmé la présence d'un **Trojan** dans ce fichier.
- L'administrateur a été immédiatement notifié et le fichier supprimé.

8. Rôle du pare-feu iptables dans le projet IDS

8.1 Objectif d'iptables

Iptables est un outil de filtrage de paquets pour les systèmes Linux. Dans notre projet, nous avons utilisé **iptables** pour :

- Protéger le SIEM et les agents contre des connexions non autorisées.
- Bloquer ou limiter le trafic malveillant en amont.
- Journaliser certaines requêtes spécifiques.

8.2 Intégration dans notre architecture

Le pare-feu iptables a été configuré sur la même machine que **Suricata**, mais aussi sur le honeypot DVWA pour un **contrôle granulaire** du trafic.

Exemples de règles appliquées :

```
# Accepter les connexions internes SSH
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.0/24 -j ACCEPT

# Bloquer les IP ayant tenté plus de 5 connexions échouées via SSH
iptables -A INPUT -p tcp --dport 22 -m recent --name blacklist --
update --seconds 60 --hitcount 5 -j DROP
```

8.3 Complémentarité avec Suricata et Wazuh

- **Suricata** détecte les attaques réseau en temps réel.
- **Wazuh** reçoit les logs système (dont iptables si configuré) pour les corréler.
- **iptables**, quant à lui, agit en **prévention** (filtrage + blocage) avant même que le trafic atteigne une application vulnérable ou le système.
- Par exemple, lorsqu'une tentative de scan est détectée, iptables peut bloquer dynamiquement l'IP (en combinaison avec une règle active Wazuh si souhaité).

VIII. Intégration de l'envoi d'e-mails

Dans le cadre de notre système de détection d'intrusions (IDS) basé sur Wazuh et Suricata, nous avons intégré une fonctionnalité d'alerte par e-mail automatique afin de notifier l'administrateur en cas de détection d'attaques critiques, comme les attaques XSS (Cross-Site Scripting).

Étapes de l'intégration :

1. Création d'une règle personnalisée dans Wazuh (local_rules.xml) :

Une règle spécifique a été ajoutée pour détecter les alertes XSS générées par Suricata. Lorsqu'un événement avec un sid (Signature ID) correspondant est détecté, la règle s'active.

```
<group name="local,syslog,sshd,">
  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <rule id="1000100" level="10">
    <if_sid>1000010</if_sid>
    <description>Suricata - Alerte XSS détectée (CUSTOM XSS)</description>
    <group>xss,web-attack,suricata</group>
    <mail_alert>yes</mail_alert>
  </rule>
</group>
```

2. Définition d'une réponse active (Active Response) :

Nous avons configuré une action automatique via le fichier ossec.conf. Cette action appelle un script Python lorsqu'une alerte critique (niveau 10) est détectée.

```
<command>
  <name>send_email_xss</name>
  <executable>send_email_xss.py</executable>
  <timeout_allowed>no</timeout_allowed>
</command>

<active-response>
  <command>send_email_xss</command>
  <location>manager</location>
  <level>10</level>
  <rules_id>1000100</rules_id>
</active-response>
```

3. Développement d'un script Python (send_email_xss.py) :

Ce script utilise le module smtplib pour envoyer un e-mail via un serveur SMTP (par exemple Gmail). L'e-mail contient un objet clair ("🚨 Alerte IDS - Attaque XSS détectée") et un message précisant qu'une attaque a été bloquée ou détectée.

Emplacement recommandé : /var/ossec/active-response/bin/send_email_xss.py

```
python
#!/usr/bin/env python3
import smtplib
from email.mime.text import MIMEText
import sys
import json

# Email config
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
EMAIL_FROM = "tonemail@gmail.com" # Remplace par ton email
EMAIL_TO = "destinataire@gmail.com" # Remplace par ton email de réception
EMAIL_SUBJECT = "🚨 Alerte IDS - Attaque XSS détectée"
EMAIL_PASSWORD = "mot-de-passe-app" # Mot de passe d'application Gmail

def send_email(message):
    msg = MIMEText(message)
    msg['Subject'] = EMAIL_SUBJECT
    msg['From'] = EMAIL_FROM
    msg['To'] = EMAIL_TO

    with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
        server.starttls()
        server.login(EMAIL_FROM, EMAIL_PASSWORD)
        server.send_message(msg)

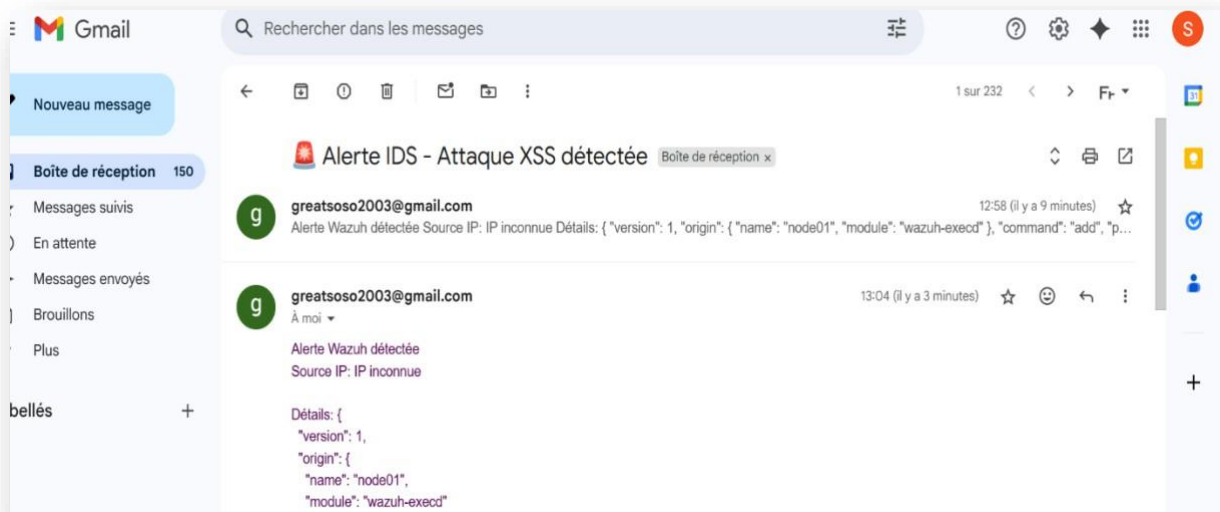
def main():
    alert_data = sys.stdin.read()
    try:
        alert_json = json.loads(alert_data.strip())
        description = alert_json.get("rule", {}).get("description", "Alerte Wazuh détectée")
        srcip = alert_json.get("srcip", "IP inconnue")
        msg = f"{description}\nSource IP: {srcip}\nDétails: {json.dumps(alert_json, indent=2)}"
        send_email(msg)
    except Exception as e:
        with open("/tmp/email_error.log", "a") as f:
            f.write(f"Erreur : {e}\n")

if __name__ == "__main__":
    main()
```

➤ Figure 37: script pour envoyer un e-mail

4. Sécurité de l'intégration :

L'adresse d'envoi utilise un mot de passe d'application Gmail (et non le mot de passe principal), ce qui augmente la sécurité. L'email du destinataire est configurable dans le script.



➤ Figure 38:e-mail reçu

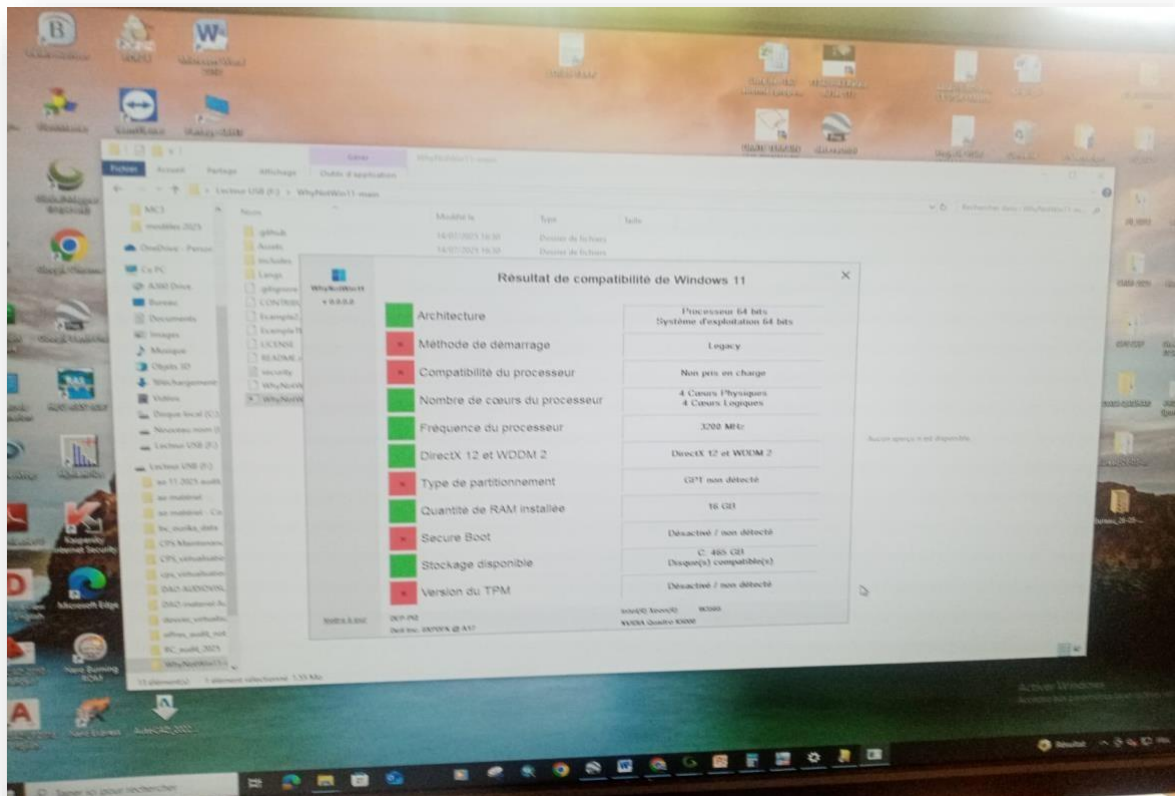
Objectif :

Cette intégration permet de **réagir rapidement** aux menaces en informant l'équipe de sécurité dès qu'un comportement suspect est détecté, sans avoir besoin de consulter constamment les journaux ou le tableau de bord.

IX. Mise en place de contrôles de sécurité essentiels sur un poste utilisateur

1. Vérification de la version de Windows installée

Nous avons commencé par vérifier la version de Windows utilisée sur la machine. Cette étape est cruciale car **Windows 10 ne sera bientôt plus pris en charge par Microsoft** (fin de support prévue en octobre 2025), ce qui signifie qu'aucune mise à jour de sécurité ne sera fournie au-delà de cette date.



➤ Figure 39: vérifié les configuration de Pc

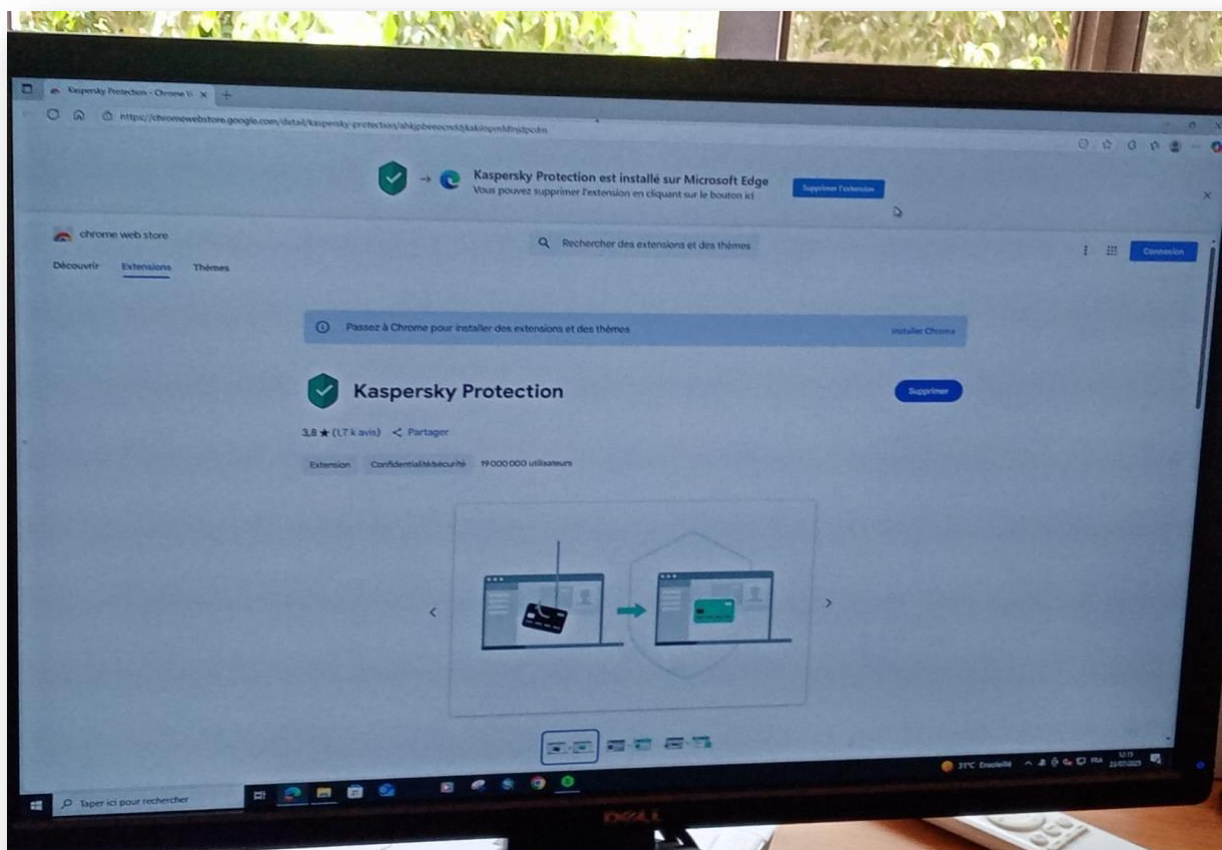
→ **Importance** : un système non supporté devient une cible facile pour les attaquants car il ne reçoit plus de correctifs de sécurité.

Nous avons également vérifié si le **PC est compatible avec Windows 11**, afin d'anticiper une éventuelle migration vers un système plus sécurisé et à jour.

2. Installation et configuration de l'antivirus Kaspersky

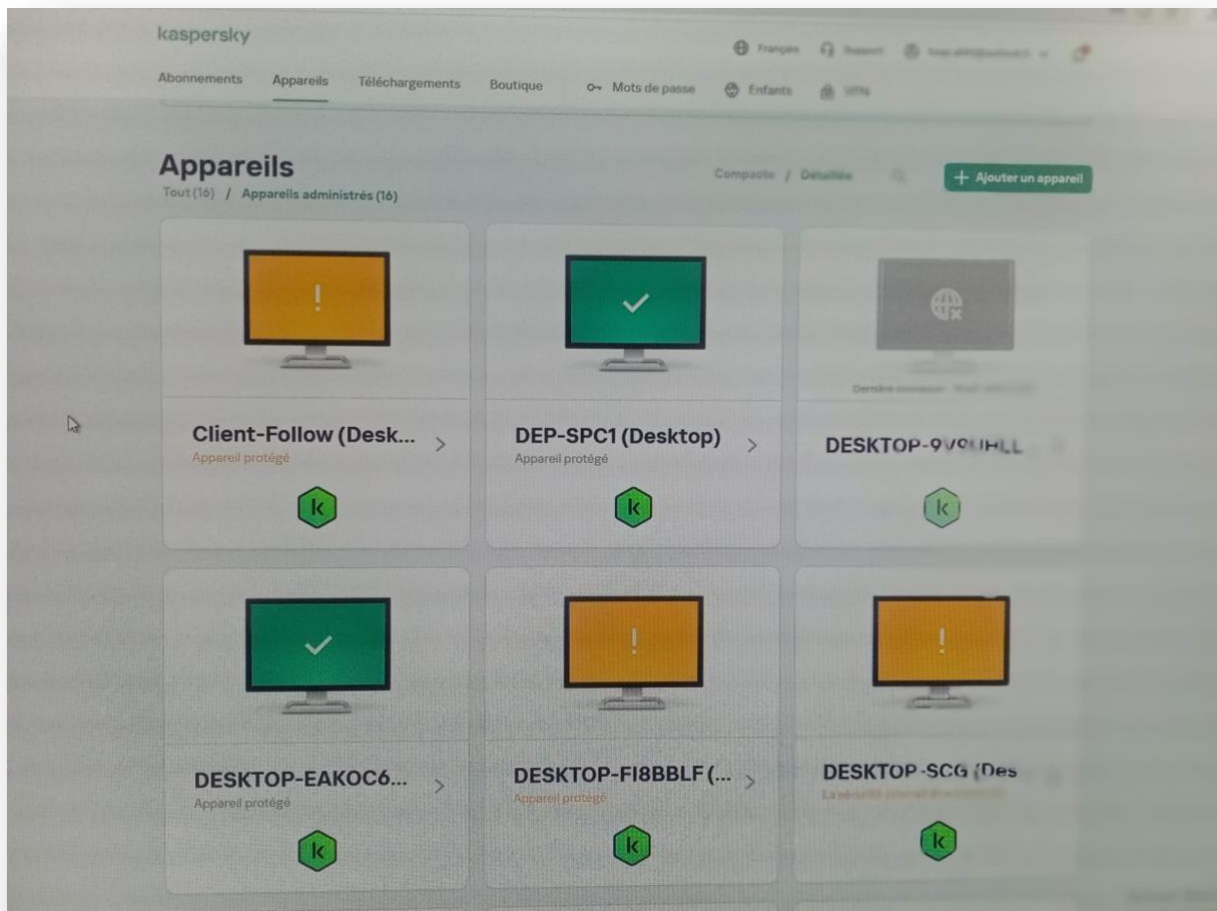
Nous avons installé **Kaspersky** (via la plateforme *My Kaspersky*) comme solution antivirus principale. Ensuite, nous avons configuré la surveillance des applications connectées, et analysé les paramètres de sécurité pour :

- **Détecter les anomalies de comportement** (exécution suspecte, consommation réseau inhabituelle).
- **Surveiller les alertes de sécurité** déclenchées par le système ou par des activités suspectes détectées.



➤ Figure 40: Installation de l'antivirus Kaspersky

→ **Importance** : l'antivirus joue un rôle central dans la **détection des malwares, ransomwares, et comportements anormaux**. Il permet de stopper une menace avant qu'elle ne compromette le système.



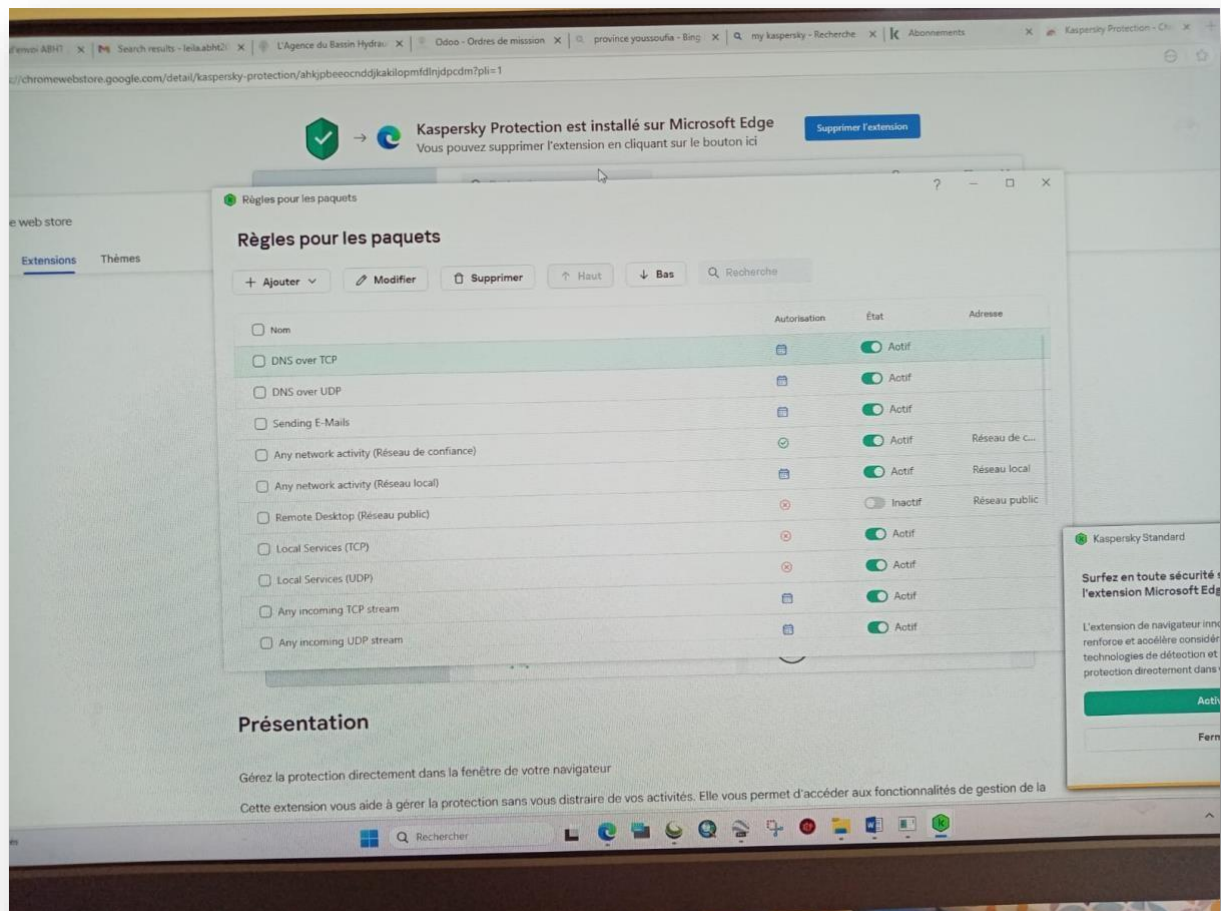
➤ Figure 41: vérifié les alerts des machines connectée

3. Compréhension du rôle de l'antivirus en tant que pare-feu

Au cours de cette étape, nous avons appris que certains antivirus (comme Kaspersky) intègrent un **pare-feu logiciel**. Nous avons exploré les fonctionnalités permettant de :

- **Bloquer ou autoriser certains protocoles réseau** (par exemple, bloquer les connexions entrantes sur des ports sensibles).

- **Bloquer des adresses IP spécifiques** considérées comme suspectes ou malveillantes.
- **Créer des règles de filtrage réseau personnalisées** selon les besoins de sécurité de l'utilisateur.



➤ Figure 42: l'antivirus en tant que pare-feu

→ **Importance** : le pare-feu intégré renforce la sécurité réseau en **limitant les communications non autorisées** et en bloquant les connexions potentiellement dangereuses.

X. Compétences Acquisées et Défis Rencontrés

1. Compétences Acquisées :

- **Maîtrise de l'installation et configuration de Suricata** en tant que système de détection et de prévention d'intrusions (IDS/IPS), avec personnalisation des règles pour détecter différents types d'attaques (scan de ports, RFI, XSS, etc.).
- **Déploiement et gestion de Wazuh** en mode distribué (manager + agent), avec intégration réussie des logs générés par Suricata pour une corrélation avancée des événements de sécurité.
- **Configuration d'un honeypot DVWA (Damn Vulnerable Web Application)** permettant la simulation d'un environnement vulnérable pour observer les comportements malveillants réels.
- **Intégration d'APIs externes :**
 - **AbuseIPDB** : pour enrichir les alertes avec une vérification de réputation d'adresse IP suspecte.
 - **VirusTotal** : pour analyser les fichiers suspects ou les URL dans les logs, et améliorer la détection de menaces.
- **Création et gestion de règles de prévention personnalisées** dans Suricata (ex : limitation des échecs SSH, détection d'injections, scan, etc.).
- **Analyse des alertes et corrélation dans le dashboard Wazuh**, compréhension des différentes sections : sécurité, intégrité des fichiers, conformité, etc.

- **Automatisation et surveillance continue** via l'interconnexion entre les outils, sans passer par Syslog grâce à la compatibilité native Suricata/Wazuh.
- **Renforcement des compétences en réseau, en systèmes Linux et en architecture de sécurité SIEM/IDS.**

2. Défis Rencontrés :

- **Conflit lors de l'installation simultanée du Wazuh Manager et Agent** sur la même machine, ce qui a nécessité de revoir l'architecture et de répartir les rôles sur des machines distinctes.
- **Manque de détection initiale dans le dashboard Wazuh**, dû à une mauvaise configuration des logs ou des règles non déclenchées (ce qui a nécessité des tests approfondis via le honeypot).
- **Problèmes de visualisation des événements de sécurité**, notamment lors de la première intégration de Suricata avec Wazuh (absence de logs, erreurs de configuration YAML, etc.).
- **Nécessité d'ajuster les règles Suricata manuellement** pour mieux adapter la détection aux scénarios testés avec DVWA (ajout de règles spécifiques à certaines attaques web).
- **Défis techniques liés à l'utilisation des APIs (AbuseIPDB, VirusTotal)** comme la gestion du taux de requêtes, l'authentification via clé API, et le parsing des résultats JSON dans un environnement de monitoring.
- **Temps de débogage et de validation** de la bonne circulation des logs entre les composants (Suricata → Wazuh Agent → Wazuh Manager → Dashboard).

Migration et Reconfiguration

XI. Clonage d'une VM préconfigurée vers un PC physique avec Clonezilla

1. Objectif de la tâche

L'objectif de cette opération était de transférer une copie exacte d'une machine virtuelle Kali Linux vers un PC physique.

La VM était préconfigurée pour notre projet de sécurité IDS/IPS avec :

- Suricata (Système de détection et prévention d'intrusion)
- Wazuh (Gestion et analyse des événements de sécurité - SIEM)

Cette migration (appelée V2P – Virtual to Physical) nous permet de :

- Déployer notre projet sur un matériel réel pour des tests de performance.
- Éviter de réinstaller et reconfigurer tout l'environnement.
- Préserver exactement la configuration du système, y compris les outils de sécurité et paramètres réseau.

2. Définition de Clonezilla

Clonezilla est un logiciel libre et open-source de clonage et d'imagerie disque. Il permet de :



- Cloner des disques ou partitions entiers.
- Créer des images compressées pour sauvegarde.
- Restaurer des images sur le même ou un autre matériel.
- Prendre en charge plusieurs systèmes de fichiers (ext4, NTFS, FAT32, etc.).

3. Vue d'ensemble du processus

Phase 1: Créer une image depuis la VM

1. Démarrer la VM Kali avec l'ISO de Clonezilla.
2. Choisir le mode device-image.
3. Sélectionner le disque externe (ex. : HDD USB) pour stocker la sauvegarde.
4. Sauvegarder le disque complet de la VM (/dev/sda) sous forme d'image.

Phase 2: l'image sur le PC physique

1. Démarrer le PC physique avec une clé USB Clonezilla.
2. Connecter le disque externe contenant l'image de la VM.
3. Choisir restaurer un disque depuis une image.
4. Écrire l'image sur le disque interne du PC physique.
5. Si nécessaire, réparer le chargeur de démarrage GRUB pour rendre le système amorçable.

4. Points techniques à considérer

- Vérifier la compatibilité des tailles de disques : le disque cible doit être au moins aussi grand que l'espace utilisé.
- Exécuter fsck avant le clonage pour corriger les erreurs du système de fichiers.
- Prévoir l'installation de pilotes matériels spécifiques après migration.

XII. Migration et Reconfiguration du Projet IDS/IPS sur Infrastructure Physique

1. Ajustements Post-Migration

Après démarrage sur matériel physique, plusieurs ajustements ont été nécessaires :

- **Compatibilité CPU** : certaines instructions utilisées par la VM n'étaient pas présentes sur le CPU physique. Suricata a été reconfiguré pour utiliser uniquement les fonctionnalités supportées.
- **Renommage des interfaces réseau** :
Les noms d'interfaces différaient entre la VM et la machine physique (ex. eth0 → enp3s0). Les fichiers de configuration de Suricata et Wazuh ont été modifiés en conséquence.

- **Changement d'adressage IP** :

Les nouvelles adresses IP ont été mises à jour dans :

- /var/ossec/etc/ossec.conf (Manager et Agents Wazuh)
- /etc/wazuh-dashboard/opensearch_dashboards.yml (Dashboard)

- **Ré-enregistrement des agents Wazuh** :

Certains agents apparaissaient encore avec leur ancienne IP. La commande suivante a été utilisée :

```
sudo /var/ossec/bin/manage_agents
```

suivie de la mise à jour du fichier client.keys et d'un redémarrage du service.

- **Optimisation mémoire du Wazuh Indexer** :

Le service échouait au démarrage à cause d'une consommation excessive de mémoire Java. Les paramètres dans :

```
/etc/wazuh-indexer/jvm.options
```

ont été ajustés pour correspondre à la RAM disponible.

2. Avantages de la Migration vers le Matériel Physique

Cette migration a apporté plusieurs bénéfices :

- Amélioration des performances – Suppression de la surcharge de virtualisation, traitement plus rapide des paquets et indexation plus fluide.
- Stabilité accrue – Moins de dépendance à l'hyperviseur et aux ressources partagées.
- Conditions de test réalistes – Comportements réseau et matériel plus proches d'un environnement de production.
- Scalabilité – Ajout plus simple d'interfaces réseau ou de stockage.
- Fiabilité sur le long terme – Moins de risques de corruption liés aux fichiers de VM.

3. Validation Finale

Après reconfiguration, l'infrastructure IDS/IPS fonctionnait normalement.

Les vérifications effectuées :

- Détection et alerte de Suricata lors d'attaques simulées.
- Communication correcte entre les agents et le Wazuh Manager.
- Accès opérationnel au Dashboard Wazuh.
- Persistance des configurations et règles après redémarrage.

Cette migration a confirmé que notre solution IDS/IPS peut être transférée d'un environnement virtualisé vers une infrastructure physique sans perte de fonctionnalités et avec un gain notable en performances.

Clés d'Accès

I. Informations d'Accès au Projet IDS/IPS

Afin de permettre une reprise rapide du projet ou une maintenance ultérieure, voici la liste des identifiants et mots de passe utilisés dans l'ensemble de l'infrastructure.

Ces informations doivent être conservées **de manière sécurisée** et ne doivent être communiquées qu'aux personnes autorisées.

1. Machines Virtuelles et Machines Physiques

- Utilisateur par défaut : *****
- Mot de passe : *****



2. Application DVWA (Damn Vulnerable Web Application)

- URL d'accès : http://<IP_DVWA>/dvwa
- Identifiants par défaut :
 - Utilisateur : *****
 - Mot de passe :*****



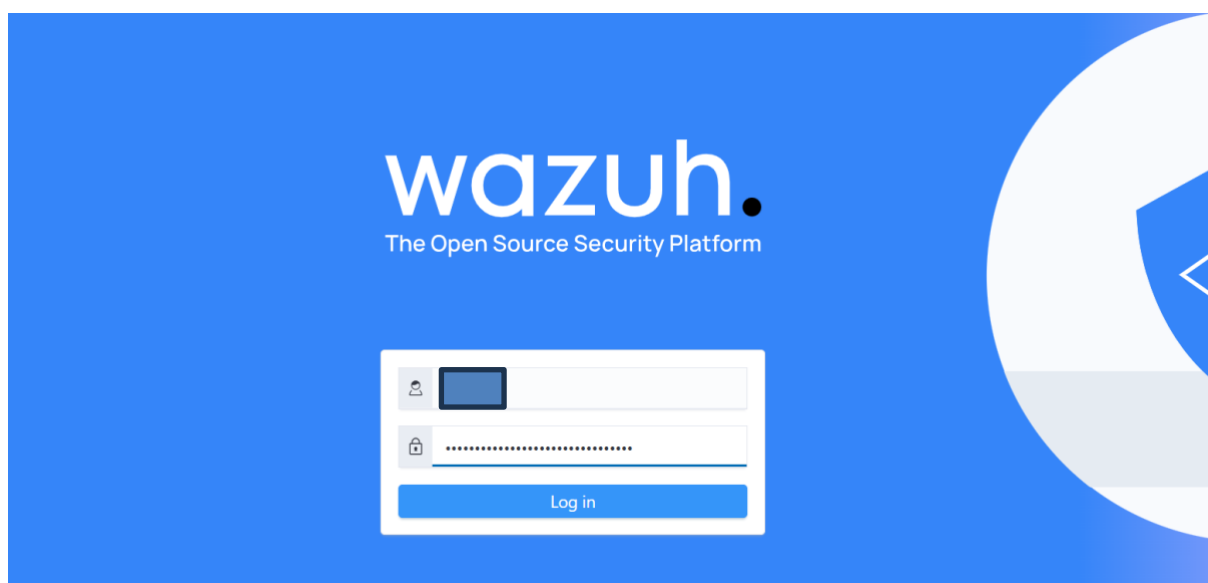
Username

Password

Login

3. Dashboard Wazuh

- URL d'accès : `https://<IP_WAZUH_DASHBOARD>`
- Compte administrateur :
 - Utilisateur : `*****`
 - Mot de passe : `*****`



4. Recommandations de Sécurité

- Modifier ces mots de passe dès que possible lors d'une mise en production réelle.
- Restreindre l'accès réseau aux interfaces d'administration.
- Stocker ces informations dans un gestionnaire de mots de passe sécurisé.

XI. RÉFÉRENCE :

- ❖ agence du bassin hydraulique tensift : <https://abht.ma/>
- ❖ Wazuh VT Integration Documentation: <https://documentation.wazuh.com/curre...>
- ❖ Official Wazuh PoC Guide: <https://documentation.wazuh.com/curre...>
- ❖ Wazuh Website: <https://wazuh.com/>
- ❖ VirusTotal Website: <https://www.virustotal.com/>
- ❖ AbuseIpDb website : <https://www.abuseipdb.com/>
- ❖ [Qu'est-ce qu'une solution SIEM \(Security Information and Event Management\) ?](#)
- ❖ [What Is SIEM ? | Security Information And Event Management Explained | How SIEM Works ? | Simplilearn - YouTube](#)
- ❖ [Integrating Suricata With Wazuh For Log Processing - YouTube](#)
- ❖ Suricata Documentation : <https://docs.suricata.io/>
- ❖ DVWA : <https://github.com/digininja/DVWA>
- ❖ AbuseIPDB API Docs : <https://docs.abuseipdb.com/>
- ❖ Tutoriel : <https://wazuh.com/blog/suricata-integration/>

Conclusion Générale :

La réalisation de ce projet de déploiement d'un système de détection d'intrusion (IDS) basé sur les technologies Suricata, Wazuh et l'intégration de l'API VirusTotal, a constitué une expérience à la fois enrichissante sur le plan technique et formatrice sur le plan professionnel. Ce projet s'inscrit pleinement dans la dynamique de renforcement de la cybersécurité adoptée par l'Agence du Bassin Hydraulique du Tensift (ABHT), dans un contexte où la protection des données critiques et des infrastructures numériques devient une priorité stratégique.

Le déploiement d'un honeypot DVWA a permis de tester l'efficacité de l'IDS dans des conditions réelles, en simulant différentes attaques web et réseau, telles que les injections SQL, les attaques XSS, les scans de ports ou encore les tentatives de brute force. L'ensemble de ces activités malveillantes a été efficacement détecté, analysé et, dans certains cas, bloqué ou supprimé automatiquement.

De plus, l'ajout de règles de surveillance d'intégrité des fichiers critiques (FIM) et la configuration d'un pare-feu iptables renforcent davantage la posture défensive du système, en agissant aussi bien en prévention qu'en réaction.

Enfin, je tiens à souligner que cette expérience m'a permis de développer des compétences concrètes en sécurité des systèmes d'information, en administration réseau, et en gestion d'un projet informatique de bout en bout, tout en contribuant à un besoin réel de l'agence.

Je remercie vivement l'équipe du département informatique de l'ABHT pour leur accompagnement, leur confiance, et leur disponibilité tout au long de ce stage.